

Agent acceleration

Making LLM agents that operate software finish a task in less time and for less money, without finishing fewer tasks.

Computer-use, web and coding agents, which work a desktop, a browser or a code base one step at a time through a model.

September 2026

PART 1

Where the time and the money go

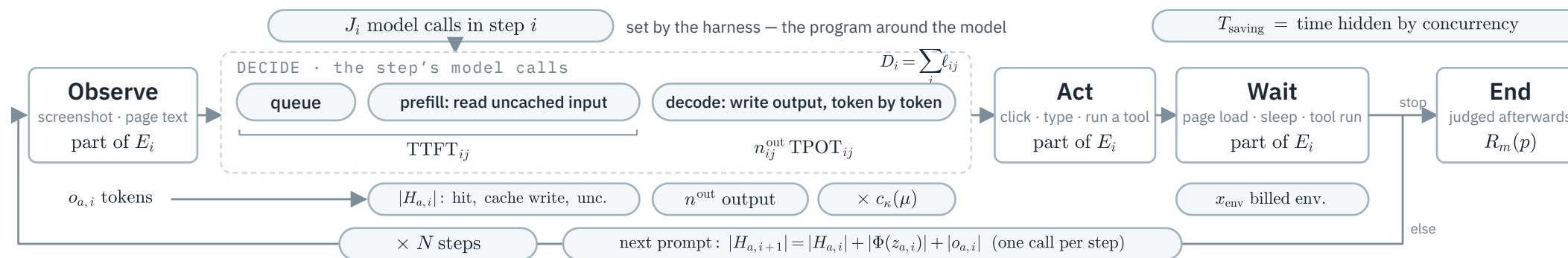
Why LLM agents that operate software are slow and expensive, and how the two are linked.

Every number comes from a published measurement of a computer-use, web or coding agent, or from a vendor’s own price page (2024–2026); the conditions of each measurement are stated next to it, and the full references are at the end of the part.

IN THIS PART

02	One attempt of an agent, step by step: the loop and its variables
03–04	The problem, defined: time, the growing prompt, money, success and the goal
05–10	Where the time goes, term by term
11–12	Where the money goes, term by term, and per success
13–14	How time and money are linked, and the levers
15–16	References · Appendix A0–A5

One attempt is N steps; each step is some model calls plus everything that is not a model call. Every variable below is defined here once; pages 3–4 add them up.



- i , N step (one observe–decide–act–wait round); steps in the attempt (i as in Chen et al., 2026; Yuan et al., 2026 · N self-defined)
- J_i , j model calls in step i (planner, judge, retries ...); call index (self-defined)
- ℓ_{ij} latency of call j of step i (ℓ as in Li et al., 2026 · indices self-defined)
- TTFT $_{ij}$ time to the first generated token: queueing, sending, prefill of the uncached input (as in Artificial Analysis, 2026)
- n_{ij}^{out} , TPOT $_{ij}$ output tokens of the call, thinking included; time per output token after the first (n as in Erol et al., 2026 · TPOT as in Chen et al., 2026)
- D_i , E_i model time of step i ; all its other time (observe, act, wait, harness gaps, back-off) (D_i adapted from Chen et al., 2026 · E_i self-defined, replacing their T_i)
- T_{saving} time hidden because some of it ran at the same time; 0 if strictly serial (adapted from Feng et al., 2026, intervals widened)
- $o_{a,i}$, a tokens the result or screenshot of step i adds to the prompt; agent index (as in Yuan et al., 2026)
- $|H_{a,i}|$, Φ , $z_{a,i}$ length of the prompt step i ’s call reads (one call per step); chat template; the call’s output (H, Φ as in Yuan et al., 2026 · z self-defined for their (θ , m, u))
- n^{hit} , n^{w} , n^{unc} input tokens read from the cache, written to it (5-min or 1-h), or uncached; $n^{\text{w}} = n^{\text{w5m}} + n^{\text{w1h}}$ (classes as billed by Anthropic, 2026b · hit as in Xu et al., 2026 · unc as in Huang et al., 2026)
- $c_{\kappa}(\mu)$, κ , μ price per token of billing class κ on serving model μ (c adapted from Erol et al., 2026 · κ , μ self-defined)
- x_{env} billed environment usage (e.g. sandbox hours) (x as in Erol et al., 2026, App. D.1 · env self-defined)
- $R_m(p)$, m , p probability that one attempt of design m (model + harness) on task p succeeds (as in Erol et al., 2026)
- reading, writing reading = prefill of input tokens; writing = decoding output tokens (a cache write is storage) (self-defined wording)

The problem, defined (1/2): the time of one attempt, and the growing prompt

from the loop → here: time as a sum over steps and calls, and why the prompt grows → to: money, success and the goal

each row says whether it is **verbatim** from its source, **adapted** (reason in Appendix A0) or **our distillation** (no source writes it as a formula) · calc. = our algebra

TIME ONE ATTEMPT adapted · our distillation	$T_{\text{attempt}} = \sum_{i=0}^N (D_i + E_i) - T_{\text{saving}}, \quad D_i = \sum_{j=1}^{J_i} \ell_{ij}$ $\ell_{ij} \approx \text{TFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}$ $T_{\text{saving}} \triangleq \mathcal{S}(\mathcal{M}) + \mathcal{S}(\mathcal{E}) - \mathcal{D}(\mathcal{M} \cup \mathcal{E})$	T_{attempt} : wall-clock of one attempt, from hand-over to result. Model and non-model time add over the steps ($i=0$ holds calls outside any step). $\triangleq$: defined as; $\mathcal{S}$: summed length of intervals; $\mathcal{D}$: length of their union (not the model time D_i); $\mathcal{M}, \mathcal{E}$: model-call and non-model intervals of the trace. sum: Chen et al., 2026 · calls within a step: our distillation, evidence Abhyankar, Qi & Zhang, 2026 · call: Artificial Analysis, 2026 (third-party) · saving: Feng et al., 2026, intervals widened
PROMPT GROWS adapted · our distillation (calc.)	$H_{a,i+1} = H_{a,i} + \Phi(z_{a,i}) + o_{a,i}$ $\sum_{i=1}^N H_{a,i} = N H_{a,1} + \frac{N(N-1)}{2} \bar{g}$	Each step appends its output and its result to the next prompt (one call per step, full history kept). $\bar{g}$: the added tokens per step, weighted by how often they are re-read. The N^2 term dominates beyond $N^* = 2 H_{a,1} /\bar{g} + 1$ (calc.). recurrence: Yuan et al., 2026 (lengths of its Eq. 3) · sum: our distillation (calc.) · re-sent history, measured: Abhyankar, Qi & Zhang, 2026

The problem, defined (2/2): the money of one attempt, per success, and the goal

from time and the prompt → here: money, cost and time per success, and what counts as acceleration → to: where the time goes

verbatim · adapted · our distillation, as on page 3 · calc. = our algebra

MONEY
ONE ATTEMPT
adapted

$$c_m(p) = \sum_{i=0}^N \sum_{j=1}^{J_i} \sum_{\kappa \in K} n_{ij}^{\kappa} c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}}$$
$$K = \{\text{hit, w5m, w1h, unc, out}\}$$

SUCCESS
PER SUCCESS
verbatim ·
adapted

$$v(m, p) = \frac{C_m(p)}{R_m(p)}, \quad \frac{\mathbb{E}[T_{\text{attempt}}]}{R_m(p)}$$

GOAL
ACCELERATION
our
distillation

$$\text{Pareto}_m \left(\frac{\mathbb{E}[T_{\text{attempt}}]}{R_m(p)}, v(m, p) \right) \quad \text{s. t.} \quad R_m(p) \geq R_0$$

$c_m(p)$: dollars of one attempt of design m (model and harness) on task p . n_{ij}^{κ} : tokens of class κ in call j of step i — cache hit, 5-min or 1-h cache write, uncached input, output. c_{env} : price per unit of billed environment usage.
base: Erol et al., 2026, Eq. 13, extended as its App. D.1 allows · classes: Anthropic, 2026b · three-class precedent: Xu et al., 2026

$v(m, p)$: expected dollars per success; the second term: expected seconds per success. $C_m(p)$: expected $c_m(p)$ over all attempts, failures included; $\mathbb{E}$: expectation. Independent attempts, retried one after another until a verified success ($1/R_m(p)$ on average); the verifier’s time and cost count in every attempt.
money: Erol et al., 2026, Eq. 2 (verbatim) · time: same derivation in seconds, as its App. D.1 allows (adapted)

Among designs that succeed at least R_0 of the time, keep those no other design beats on both time and money per success; the reference is a person doing the task today. Per task p ; over a task mix,
 $\sum_p C_m(p) / \sum_p R_m(p)$.
distilled from Kapoor et al., 2025 (“jointly optimizing the two metrics”, arXiv v1) and Erol et al., 2026 (App. C.8: leave unreliable systems off the frontier)

WHAT THE FORMULAS DO NOT SHOW: THE VARIABLES ARE COUPLED

- The prompt length drives both sides: prefill time and the input bill (Abhyankar, Qi & Zhang, 2026)
- The harness and provider, not the model, set the cache split (Zhu et al., 2026)
- Time can cost money: a cache entry expires after 5 min or 1 h, so a slow step can turn cache reads into cache writes (our reading of Anthropic, 2026b)
- Thinking lengthens D_i and the output bill; calls and thinking change $R_m(p)$ (page 13)

Where the time goes: five classes, one per term

from the definition → here: the terms of the time of one attempt, each with one measurement → to: steps and calls

Four of the five are terms of the time equation; the fifth is how they combine: the concurrency term T_{saving} is small in the one production trace that reports it, so the terms nearly add. Which class dominates depends on the harness, not on the kind of agent (page 10). Cause-by-cause tables with measurement conditions: Appendix A1.

$$T_{\text{attempt}} \approx \left[\sum_i \left[\sum_{j \leq J_i} (\text{TTFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}) + E_i \right] - T_{\text{saving}} \right]$$

I · STEPS II · CALLS PER STEP III · EACH CALL IV · NON-MODEL TIME V · LITTLE RUNS CONCURRENTLY

I · TOO MANY STEPS (N)

Hour-scale tasks take hundreds of steps, and every step is a full round trip

- 318 tool calls per task on OSWorld 2.0 — 108 desktop tasks, Claude Opus 4.7, one action per step

(XLANG Lab, 2026)

II · TOO MANY CALLS PER STEP (J_i)

A harness that plans, judges and reflects multiplies every step

- 4–12 planning calls per judging call — GTA1 harness: four parallel planners, up to three retries, one judge

(Abhyankar, Qi & Zhang, 2026)

III · EACH CALL IS SLOW (ℓ_{ij})

Inside a call, waiting, reading and writing are three separate delays

- Writing, one token at a time, is 91–98.6% of model time — local 27–31B models with a warm cache

(Yuan et al., 2026)

IV · NON-MODEL TIME (E_i)

The environment costs time whether or not the model is running

- 6.6 s per browser action vs 4.7 s per model call, median — 151 WebVoyager live-site tasks, GPT-4o

(Wong et al., 2026)

V · LITTLE RUNS CONCURRENTLY (T_{saving})

Almost nothing overlaps, so task time is close to a sum: fixing one term saves only its own share

- Concurrency within a turn: 1.15 — GitHub Copilot production telemetry

(Liu et al., 2026)

I–II · Too many steps, too many calls per step

from the five classes → here: the two sums, steps and calls per step → to: what happens inside a call

N multiplies every term, and J_i multiplies N . Steps accumulate because tasks are long, each step does one action, navigation-only steps still go through the model, and the agent idles or loops; calls accumulate because the harness plans, judges and reflects, or samples several candidates per step.

I-1, I-2 · LONG TASKS, ONE ACTION PER STEP

Long tasks with one action per step need hundreds of steps; several actions per call cut them by half or more

- 318 tool calls per task with one action per step, 160.7 with several actions per call — OSWorld 2.0, 108 long tasks, Claude Opus 4.7

(XLANG Lab, 2026)

I-3 · NAVIGATION-ONLY STEPS

Most steps are navigation that needs no thinking, yet each one is a model call

- 66.7% of the steps in the median task are pure navigation — 151 WebVoyager tasks on live websites, three text agents driven by GPT-4o

(Wong et al., 2026)

I-4 · IDLING AND DEAD LOOPS

When an agent is stuck, every repeated step is billed and timed in full

- One element-locating loop repeated a step 18 times: 27 minutes and \$8.47 at list price without caching — GTA1 harness, OSWorld

(Abhyankar, Qi & Zhang, 2026)

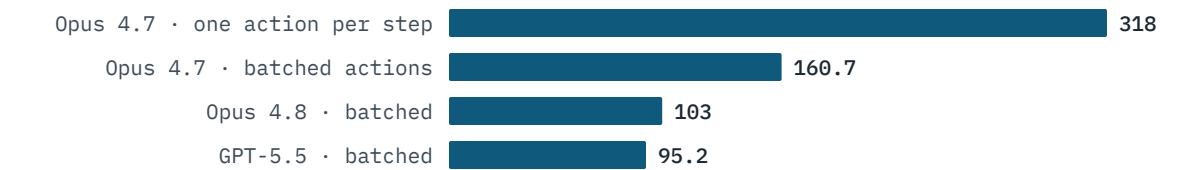
II · PLANNING, JUDGING, SAMPLING

Extra calls per step buy accuracy at a rising token cost per point

- 10 candidates per step instead of 1: 38.8% → 43.2% success for 96K → 920K tokens per task — gpt-oss-120b, 165 WebArena-Lite tasks

(Lee et al., 2026)

Steps per task on OSWorld 2.0, 108 tasks



XLANG Lab, 2026 · mean over tasks

III · Inside one call: first token, then decoding

from the two sums → here: the latency of one call, TTFT plus decoding → to: why the prompt keeps growing

A call waits in a queue, reads its uncached input, then writes its output one token at a time: $\ell_{ij} \approx \text{TTFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}$. Queueing depends on the provider, reading on the prompt, writing on how much the model writes, thinking included.

III-1 · QUEUEING

The same request can wait far longer depending on the provider's load

- Identical requests take up to 69× longer depending on the time of day — 15 models, 5 providers

(Bian et al., 2025)

III-2 · PREFILL

For screenshot agents, reading the prompt dominates a call, and the prompt grows every step

- Planning, judging and reflection latency is “dominated by the prefill stage” — GTA1 and Agent S2 harnesses, 39 OSWorld tasks

(Abhyankar, Qi & Zhang, 2026)

III-3 · DECODING

Writing is the slowest segment per token, and thinking modes write far more tokens

- 224K vs 37K output tokens per task — Claude Opus 4.8 vs GPT-5.5 on the same 108 OSWorld 2.0 tasks
- Writing is 91–98.6% of model time — local 27–31B models with a warm cache

(XLANG Lab, 2026; Yuan et al., 2026)

HOW TTFT IS MEASURED

Published first-token times lump queueing, network and prefill together

- Time to first token is measured from the request being sent and “includes network latency”; output speed excludes the first chunk

(Artificial Analysis, 2026)

The prompt grows: every step re-reads the whole history

from one call → here: the prompt row of page 3, measured → to: the time outside the model

Each step appends its output and its result, so the tokens read over an attempt grow with N^2 ; caching makes re-reading cheaper and faster, not free. The square matters once N passes a few dozen steps.

HISTORY RE-SENT

Most computer-use agents send the whole history at every step

- “At each step, the prompt sent to the LLM includes the history of all previous steps”; later steps take up to 3× longer — GTA1 harness, 39 OSWorld tasks

(Abhyankar, Qi & Zhang, 2026)

SCREENSHOTS ARE THE BULK

One screenshot adds about a thousand tokens to every later prompt

- A 1280×720 screenshot is $46 \times 26 = 1,196$ tokens on Claude, one token per 28×28-pixel patch (calc.)

(Anthropic, 2026b)

CACHED, STILL PAID

A high cache hit rate lowers the price of re-reading, but the prefix stays the larger part of the bill

- 95.7% hit rate, yet prefix tokens are 59.5% of the list-price cost — 4,265 Claude Code / Codex sessions

(Zhu et al., 2026)

WHEN THE SQUARE BITES

Below a few dozen steps the first prompt dominates; long tasks are deep in the square

- $N^* = 2|H_{a,1}|/\bar{g} + 1$: a 6,000-token first prompt and 1,500 tokens added per step give $N^* = 9$ (calc., assumed values); OSWorld 2.0 tasks take about 318 steps

(XLANG Lab, 2026)

Per step, the pressure on prefill is the appended part, not the whole prompt: with a warm cache, “the relevant per-turn prefill pressure is better captured by incremental appends” (Yuan et al., 2026).

IV–V · The time outside the model, and how little of it overlaps

from the prompt → here: non-model time and concurrency → to: which term dominates

The environment is slow, and today's agents run almost everything in sequence, so T_{saving} is small and the attempt time is close to the full sum. Published “total times” differ in what they count, so the boundary has to be stated.

IV · NON-MODEL TIME

The environment is slow on its own, and the benchmarks add fixed sleeps on top

- OSWorld 2.0 sleeps 3 s after every action: × 318 steps ≈ 16 min of pure waiting per task (calc.)
- 6.6 s per browser action vs 4.7 s per model call, median — 151 WebVoyager live-site tasks, GPT-4o

(XLANG Lab, 2026; xlang-ai, 2026; Wong et al., 2026)

V · CONCURRENCY

Almost nothing overlaps, so fixing one term saves only its own share

- Concurrency within a turn: 1.15 — GitHub Copilot production telemetry

(Liu et al., 2026)

Where a Claude Code / Codex request’s 4.3 minutes go



Zhu et al., 2026 · 4,265 sessions, 43 developers

WHAT A “TOTAL TIME” COUNTS

Some published times include the person, others exclude them

- 80–92% of a coding session’s elapsed time is the person thinking between turns, outside any one attempt

(Liu et al., 2026; Zhu et al., 2026)

Which term dominates depends on the harness, not on the kind of agent

from the five terms → here: their shares, per kind of agent → to: where the money goes

All five classes exist in every agent; the heaviest term differs, and it flips when the harness changes.

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
HEAVIEST	Reading × calls per step × steps. 87–97% of task time is planning, judging and reflection calls (GTA1, Agent S2; 39 OSWorld tasks) (Abhyankar, Qi & Zhang, 2026)	Browser execution and waiting. Per step, median: browser 6.6 s, model 4.7 s (151 WebVoyager live-site tasks, GPT-4o) (Wong et al., 2026)	Writing, or tool tails. Writing is 91–98.6% of model time (Yuan et al., 2026); in Claude Code / Codex requests tools take 59.8%, the model 41.0% (Zhu et al., 2026)
FLIPS WHEN	The harness makes one call per step: over 70% of the time is then the sandbox, because the benchmark sleeps 2–3 s after every action (Chang et al., 2026; xlang-ai, 2026; XLANG Lab, 2026)	The observation grows: 4.8× the input moved the model’s share of step time from 46.9% to 61.6% (Chang et al., 2026)	Load evicts the cache: latency up to 7.14× (Kang et al., 2026)
IN ONE LINE	Slow in reading: every step re-reads the growing screenshot history	Slow in the environment — until the observation grows, then slow in reading	Reading is cached away; slow in writing and in the tools

1 · The harness sets the dominant term — calls per step, observation size, fixed sleeps, cache on or off. The explanation “agents are slow because inference is slow” is accurate only for multi-call screenshot agents.

2 · N multiplies every term — one step fewer saves a whole step of time and money in all three kinds.

Where the money goes: six classes, one per term

from where the time goes → here: the terms of money per success, each with one measurement → to: per attempt vs per success

Money comes from tokens read and tokens written, each times a price, plus billed environment usage, which is usually small. Writing costs 5× reading; a cached read costs a small fraction of a normal read (DeepSeek-V4.1-Flash: \$0.006 vs \$0.30 per million tokens); fast mode costs 2× (Anthropic, 2026b; OpenAI, 2026; DeepSeek, 2026).

$$v(m, p) = \left[\mathbb{E} \left[\sum_i \sum_{j \leq J_i} \right] \right. \left. \sum_{\kappa} n_{ij}^{\kappa} \right] \left[c_{\kappa}(\mu_{ij}) \right] \left[+ x_{\text{env}} c_{\text{env}} \right] \left[\div R_m(p) \right]$$

3 · NUMBER OF CALLS

1, 2 · TOKENS BY CLASS

5 · PRICE PER CLASS

6 · ENVIRONMENT

4 · FAILURES, RETRIES

1 · TOKENS READ

Reading is the larger bill and grows with the square of the steps

- Even at a 95.7% cache hit rate, prefix tokens are still 59.5% of the bill — 4,265 Claude Code / Codex sessions

(Zhu et al., 2026)

2 · TOKENS WRITTEN

Writing is the dearer token, thinking included, but the smaller bill

- 31% of an uncached screenshot agent’s bill: \$2.43 counting output only vs \$7.87 counting all tokens — GTA1, 39 OSWorld tasks (calc.)

(Abhyankar, Qi & Zhang, 2026)

3 · NUMBER OF CALLS

The calls that make the agent slow also make it expensive

- 5–13× the calls of a single-call harness per step — GTA1: planners, retries and a judge (calc.)

(Abhyankar, Qi & Zhang, 2026)

4 · FAILURES AND RETRIES

Failed attempts are billed in full

- \$72.4 per attempt ÷ 20.6% completion ≈ \$351 per success — OSWorld 2.0, best agent (calc.)

(XLANG Lab, 2026)

5 · UNIT PRICE

The price list sets the constant; fast tiers double it

- Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written — a 40× spread (calc.)

(Anthropic, 2026b; OpenAI, 2026)

6 · THE ENVIRONMENT MACHINE

Published costs are API bills only; a CPU environment adds little

- \$0.02–0.36 for a one-hour CPU environment: 0.3–4.6% of a \$7.87 bill (calc.; next page)

vendor price pages, read 2026-09-27

Four accounting conventions for one task, and the cost of the environment machine

from the six classes → here: per attempt vs per success, which convention a figure uses, and the environment → to: how time and money are linked

The same run’s cost differs by up to 30× across the four conventions in use, and none includes the environment machine.

CONVENTION	THE SAME RUN, COUNTED BOTH WAYS	GAP
OUTPUT ONLY VS ALL TOKENS	GTA1 on 39 OSWorld tasks, o3 list price, no cache: \$2.43 counting output only; \$7.87 counting all tokens (Abhyankar, Qi & Zhang, 2026)	3.2×
PER ATTEMPT VS PER SUCCESS	OSWorld 2.0, Claude Opus 4.8: \$72.4 per attempt at 20.6% completion → ≈ \$351 (calc.; a benchmark average) (XLANG Lab, 2026)	4.9×
UNCACHED VS CACHED	A cached read costs a fraction of a normal read (DeepSeek-V4.1-Flash: \$0.006 vs \$0.30 per million tokens; other vendors in A3) (DeepSeek, 2026)	4–50× on the read price only, by vendor (calc.)
STANDARD VS FAST MODE	Claude Opus 5.5 \$4 / \$20 → \$8 / \$40; OpenAI 2× on every listed model; up to 2.5× faster writing (vendor-stated), reading unchanged (Anthropic, 2026b; OpenAI, 2026)	2×

The three that apply to a whole bill compound to about 31× (3.2 × 4.9 × 2, calc.). Screenshot-agent measurements do not report their cache state.

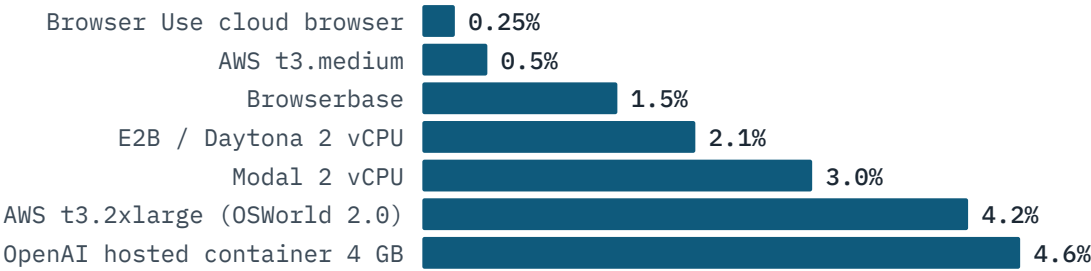
6 · THE ENVIRONMENT MACHINE, FROM LIST PRICES

For CPU environments the machine is a small addition; idle time, proxy traffic and GPUs change that

- An H100 sandbox at \$3.95 per hour is already half of a \$7.87 bill, and hourly billing runs while the agent waits — the median AgentSysBench session is active only 20% of its lifetime

vendor price pages, read 2026-09-27; Chang et al., 2026

A one-hour CPU environment as a share of a \$7.87 API bill (calc.)



vendor price pages, read 2026-09-27 · full price table in Appendix A3

How slow and expensive are linked: three kinds of relationship

from the two sides → here: what each term does to time and to money → to: the levers

Time and money share the token counts; queueing and environment time reach money only through billed environment usage — and through the cache, which expires while the agent waits.

TIME

≈

$\sum_i \left[\sum_j \left(t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \text{TPOT}_{ij} \right) + E_i \right] - T_{\text{saving}}$

MONEY

=

$\sum_i \sum_j \sum_{\kappa} n_{ij}^{\kappa} c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}}$

in both — same source

time only, or money only through billed environment usage

the price — money for speed or accuracy

t_{ij}^{queue} , t_{ij}^{prefill} the queueing and prefill parts of TFTT_{ij} (our split; network and first-token time left out; Appendix A0)

time, money one attempt each; divide by $R_m(p)$ for per success

SAME SOURCE — IN BOTH EQUATIONS

fix it, and time and money fall together

- Steps N , calls per step J_i , reading, writing, cache hits, failures — every one of them lives in the token counts

(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026b)

SLOW BUT NOT EXPENSIVE — NO TOKENS, ONLY MACHINE HOURS

fix it, and only time falls

- Queueing, page loads, fixed sleeps, tool tails — none produces a token; the machine behind them is billed by the hour, usually under 5% of the API bill

MONEY FOR TIME OR FOR ACCURACY — OPPOSITE SIGNS

paying for speed shortens only the writing segment; paying for accuracy costs more tokens per point as accuracy rises

- Fast mode: up to 2.5× faster writing (vendor-stated) at 2× the price, reading unchanged

(Anthropic, 2026b; OpenAI, 2026; accuracy: Appendix A4)

TIME THAT COSTS MONEY — THE CACHE EXPIRES

a slow step or a long pause can turn cheap cache reads into cache writes

- A cache entry lives 5 minutes or 1 hour from the start of the last request that read or wrote it; at most 12.8% of cost could be saved if the cache survived human pauses

(Anthropic, 2026b; Zhu et al., 2026)

The levers, and what is still unmeasured

from how time and money are linked → here: what fixing each lever buys, and the holes in the evidence → to: Part 2, what the literature has done with each lever

Seven levers follow from the formulas; each moves one term and carries a known risk. Part 2 asks how far the literature has pushed each. Right: gaps in the evidence, not claims of a research gap.

LEVER	TIME	MONEY	THE CATCH
Fewer steps N — several actions per call, skip navigation-only steps	yes	yes, the quadratic term	which steps need no thinking?
Read less $ H_{a,i} $, n^{unc} — trim history, shrink screenshots, cache the prefix	yes	yes	forgets; trimming invalidates the cache
Write less n^{out} — less thinking, smaller model	yes	yes	accuracy may fall
Fewer calls per step J_i — drop judging and reflection	yes	yes	fewer errors caught
Overlap the waiting T_{saving} — pre-load, parallel tools, page-ready events	yes	no	only where the environment dominates
Buy a fast or priority tier TPOT, c_κ	writing only	no — 2× more	reading unchanged; cache dropped
Stop retrying failures $R_m(p)$ — early stop, detect dead loops	yes	yes	stops some attempts that would have succeeded

NOT YET MEASURED

- **Read / write time split and cache hit rate for a frontier API model** — the only split is on local 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state.
- **How much slower than a person** — no benchmark times agent and human on the same tasks; the closest, AXIS’s small user study, has a UI agent 1.69× slower than manual work on easy tasks (Lu et al., 2025).
- **Whether the benchmark’s fixed sleep sits inside “action” time** in OSWorld-Human — if so, “environment under 3.5%” understates the waiting.
- **How much slowness costs accuracy** — OSWorld 2.0 records the stale-screen failure mode without its share (XLANG Lab, 2026).

References · Part 1 (1 of 2)

author-year tags on the pages refer to these entries · published version where one exists, otherwise the arXiv number and the authors' institutions

1. Abhyankar, R., Qi, Q., & Zhang, Y. (2026). OSWorld-Human: Benchmarking the efficiency of computer-use agents. *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys 2026)*. arXiv:2506.16042. University of California, San Diego.
2. Anthropic. (2026a, May 13). *Best practices for computer and browser use with Claude*. <https://claude.com/blog/best-practices-for-computer-and-browser-use-with-claude>
3. Anthropic. (2026b). *Claude API pricing; Prompt caching; Fast mode; Vision* (developer documentation, read 28 September 2026). <https://platform.claude.com/docs/en/about-claude/pricing>; <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>; <https://platform.claude.com/docs/en/build-with-claude/fast-mode>; <https://platform.claude.com/docs/en/build-with-claude/vision>
4. Artificial Analysis. (2026). *Methodology; Performance benchmarking methodology* (read 28 September 2026). <https://artificialanalysis.ai/methodology>; <https://artificialanalysis.ai/methodology/performance-benchmarking> — third-party measurement definitions.
5. Bian, S., Yan, M., Jayarajan, A., Pekhimenko, G., & Venkataraman, S. (2025). What limits agentic systems efficiency? arXiv:2510.16276. University of Wisconsin–Madison; University of Toronto; NVIDIA.
6. Chang, C., Zhou, Y., Fu, K., An, D., Feng, T., Lu, H., Yao, S., Guo, P., Yu, Y., Shan, Y., Li, B., Yuan, B., & Wang, W. (2026). From LLM inference to agentic workloads: Characterization and implications for serving systems (AgentSysBench). arXiv:2608.15127. Hong Kong University of Science and Technology; Alibaba Group; ByteDance.
7. Chen, H. M., Guo, J., Luk, W., & Fan, H. (2026). AOSpec: Action and observation co-speculation for low-latency agent serving. arXiv:2608.00881. Imperial College London.
8. Chen, L., Zaharia, M., & Zou, J. (2024). FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. arXiv:2305.05176. Stanford University.
9. DeepSeek. (2026). *Models & pricing* (API documentation, read 28 September 2026); *DeepSeek-V4.1-Flash release* (10 September 2026). https://api-docs.deepseek.com/quick_start/pricing; <https://api-docs.deepseek.com/news/news260910>
10. Erol, M. H., El, B., Suzgun, M., Yuksekogul, M., & Zou, J. (2026). Cost-of-Pass: An economic framework for evaluating language models. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2504.13359v2. Stanford University.
11. Feng, G., Mao, H., Dutta, P., & Gonzalez, J. E. (2026). Concurrency without model changes: Future-based asynchronous function calling for LLMs (AsyncFC). arXiv:2605.15077. University of California, Berkeley.
12. Google. (2026). *Gemini Developer API pricing* (read 28 September 2026). <https://ai.google.dev/gemini-api/docs/pricing>
13. Huang, Z., Wang, X., Wang, A., Jurayj, W., Jiménez Gutiérrez, B., Khashabi, D., & Andrews, N. (2026). Better, faster, stronger: Programmatic skill learning best reduces agent cost (SpeedRunner). arXiv:2608.11338v1. Johns Hopkins University.
14. Kang, H., Li, Z., Yang, X., Xu, W., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., & Arora, S. (2026). ThunderAgent: A fast, simple, and program-aware agentic inference system. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, poster. arXiv:2602.13692. Georgia Institute of Technology; University of Illinois Urbana-Champaign; Carnegie Mellon University; Together AI.
15. Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2025). AI agents that matter. *Transactions on Machine Learning Research*. arXiv:2407.01502. Princeton University.
16. Kapoor, S., Stroebl, B., Kirgis, P., Nadgir, N., Siegel, Z. S., Wei, B., ... Narayanan, A. (2026). Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2510.11977. Princeton University et al.
17. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., & Gholami, A. (2024). An LLM compiler for parallel function calling. *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*. arXiv:2312.04511. University of California, Berkeley.
18. Le Sellier de Chezelles, T., Gasse, M., Lacoste, A., Caccia, M., Drouin, A., Boisvert, L., ... Chapados, N. (2025). The BrowserGym ecosystem for web agent research. *Transactions on Machine Learning Research*. arXiv:2412.05467. ServiceNow Research et al. (figure not re-verified; used only in Appendix A1)

References · Part 1 (2 of 2)

continued

19. Lee, N., Erdogan, L. E., John, C. J., Krishnapillai, S., Mahoney, M. W., Keutzer, K., & Gholami, A. (2026). Agentic test-time scaling for WebAgents. arXiv:2602.12276. University of California, Berkeley.
20. Li, Y., Ye, Q., Choubey, P. K., Zhang, J., & Wu, C.-S. (2026). Speculate with memory. arXiv:2607.12236. Salesforce Research.
21. Liu, B., Qiu, H., Goiri, Í., Fonseca, R., Bianchini, R., & Choukse, E. (2026). Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. arXiv:2608.00101. University of Illinois Urbana-Champaign; Microsoft Azure Research.
22. Lu, J., Zhang, Z., Yang, F., Zhang, J., Wang, L., Du, C., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). AXIS: Efficient human-agent-computer interaction with API-first LLM-based agents. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7711–7743. <https://doi.org/10.18653/v1/2025.acl-long.381>. Microsoft.
23. Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., & Stoica, I. (2026). Agentix: An efficient serving engine for LLM agents as general programs. *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. University of California, Berkeley; Google DeepMind; Shanghai Jiao Tong University.
24. OpenAI. (2026). *API pricing; API changelog* (fast mode, 30 July 2026; Ultrafast, 13 August 2026) (read 28 September 2026). <https://developers.openai.com/api/docs/pricing>; <https://developers.openai.com/api/docs/changelog>
25. Vendor price pages used on pages 11–12 and in Appendix A3 (all read 27 September 2026): AWS, *Amazon EC2 T3 instances*; Browser Use, *Pricing* and *API v4: create browser session*; Browserbase, *Pricing* and *Billing plans*; Daytona, *Pricing* and *Billing*; E2B, *Pricing*; Modal, *Pricing* and *Sandbox resources*.
26. Winston, C., Wang, R. Y., Mirhoseini, A., & Kozyrakis, C. (2026). Agent JIT compilation for latency-optimizing web agent planning and scheduling. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, PMLR 306. arXiv:2605.21470. Stanford University.
27. Wong, M., Hsieh, K., Nath, S., & Netravali, R. (2026). Skim: Speculative execution for fast and efficient web agents. arXiv:2605.16565. Princeton University; Microsoft Research.
28. xAI. (2026). *Grok 4.7* model page, SpaceXAI Docs (read 28 September 2026). <https://docs.x.ai/developers/models/grok-4.7>
29. XLANG Lab. (2026). OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. arXiv:2606.29537. The University of Hong Kong (authored as “XLANG Lab and Collaborators”; the full author list is in the paper’s Appendix A).
30. xlang-ai. (2026). *OSWorld* [Source code], `desktop_env/desktop_env.py` and `run.py` at commit b138d348. <https://github.com/xlang-ai/OSWorld>
31. Xu, B., Xue, Z., Chen, D., Fu, C., Wu, C., Huang, C., ... Zhang, N. (2026). TokenPilot: Cache-efficient context management for LLM agents. arXiv:2606.17016. Zhejiang University; University of Electronic Science and Technology of China; Xidian University; HomologyAI.
32. Ye, N., Ahuja, A., Liargkovas, G., Lu, Y., Kaffes, K., & Peng, T. (2026). Speculative actions: A lossless framework for faster AI agents. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*, oral. arXiv:2510.04371. Columbia University.
33. Yuan, Y., Nayak, A., Kundu, S., & Talati, N. (2026). Agentic AI workload characteristics. arXiv:2605.26297v2 (21 September 2026). University of Illinois Urbana-Champaign; Gimlet Labs; Intel.
34. Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., ... Zhang, D. (2026). UFO2: The desktop AgentOS. *Transactions on Machine Learning Research*. arXiv:2504.14603. Microsoft.
35. Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., & Kasikci, B. (2026). TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560. University of Washington.

A0 · The time of one attempt: each source’s own formula, and why ours differs

the derivation behind page 3 · verbatim / adapted / our distillation · calc. = our algebra

FORMULA (PAGES 2-3)	STATUS	THE SOURCE’S OWN FORMULA OR WORDS, IN ITS NOTATION	WHAT CHANGED, AND WHY
$T_{\text{attempt}} \triangleq t_{\text{end}} - t_{\text{start}} = \mathcal{D}(\mathcal{M} \cup \mathcal{E}) = \sum_{i=0}^N (D_i + E_i) - T_{\text{saving}}$	adapted	T_{saving} is “defined as the difference between the serialized baseline and the observed end-to-end asynchronous latency” (App. B.2) (Feng et al., 2026), combined with the sum below	one attempt, not one task (a task may take several); exact because the non-model intervals cover every instant; $T_{\text{saving}} \geq 0$ (calc.); exact with measured call times, approximate once $\ell \approx \text{TTFT} + n \text{ TPOT}$
$\sum_{i=0}^N (D_i + E_i)$	adapted	“At step i, the actor takes D_i time to generate action a_i , and the runtime takes T_i time to execute it ... yielding serial latency $\sum_i (D_i + T_i)$ ” (§3) (Chen et al., 2026)	$T_i \rightarrow E_i$ (T is kept for totals); D_i covers all calls of a step, E_i every non-model interval, both as summed lengths; $i=0$ holds calls outside any step; AOSpec’s a_i is an action, not our agent index; preferred to (Kim et al., 2024), whose N counts the function calls of one plan
$D_i = \sum_j \ell_{ij}$	our distillation	No source sums the calls of a step. GTA1 makes “4 parallel calls” per planning step and “between 4 and 12 planning calls” per judging call (Abhyankar, Qi & Zhang, 2026)	parallel calls add here; their overlap is credited to T_{saving} ; the letter ℓ as in (Li et al., 2026)
$\ell_{ij} \approx \text{TTFT}_{ij} + n_{ij}^{\text{out}} \text{TPOT}_{ij}$	adapted	Total Response Time = Time to First Token + 100 / Output Speed (Artificial Analysis, 2026)	100 $\rightarrow$ the call’s own output tokens, thinking included; 1/speed $\rightarrow$ TPOT; $\approx$ because output speed excludes the first chunk; TTFT runs to the first token of any kind, reasoning included
$\text{TTFT}_{ij} \approx t^{\text{queue}} + t^{\text{net}} + t^{\text{prefill}} (n^{\text{unc}} + n^{\text{w}}; n^{\text{hit}}) + t^{\text{first}}$	our distillation	No source writes TTFT as a sum. A program’s latency “comprises three components”, the first the “queuing time” of its calls (Luo et al., 2026); TTFT “includes network latency” (Artificial Analysis, 2026)	the split used on page 13; t^{net} : network time, t^{first} : first decode step; the shape of t^{prefill} is not given by any verified source

A0 · Time hidden by concurrency, and the growing prompt

the derivation behind page 3 · verbatim / adapted / our distillation · calc. = our algebra

FORMULA (PAGE 3)	STATUS	THE SOURCE’S OWN FORMULA OR WORDS, IN ITS NOTATION	WHAT CHANGED, AND WHY
$T_{\text{saving}} \triangleq \mathcal{S}(\mathcal{M}) + \mathcal{S}(\mathcal{E}) - \mathcal{D}(\mathcal{M} \cup \mathcal{E})$	adapted	the same formula, continued $= \Delta_{F \parallel F} + \Delta_{D \parallel E}$, with $\mathcal{M}, \mathcal{E}$ “the sequence of time intervals for model decoding and function execution” (App. B.2) (Feng et al., 2026)	intervals widened to whole calls and all non-model time, so values are not comparable with AsyncFC’s; the split becomes (calc.) $[\mathcal{S}(\mathcal{M}) - \mathcal{D}(\mathcal{M})] + [\mathcal{S}(\mathcal{E}) - \mathcal{D}(\mathcal{E})] + [\mathcal{D}(\mathcal{M}) + \mathcal{D}(\mathcal{E}) - \mathcal{D}(\mathcal{M} \cup \mathcal{E})]$, each ≥ 0 ; it can include extra work that concurrency causes, so it is not the saving against a separate serial run (speed-ups use $T_{\text{seq}}/T_{\text{attempt}}$, Part 2)
$ H_{a,i+1} = H_{a,i} + \Phi(z_{a,i}) + o_{a,i} $	adapted	$H_{a,i+1} = H_{a,i} \parallel \Phi(\theta_{a,i}, m_{a,i}, u_{a,i}) \parallel o_{a,i}$, $C_{a,i} = H_{a,i} $ (Eqs. 1–3, v2 §II-C) (Yuan et al., 2026)	lengths of the concatenation; $z = (\theta, m, u)$ (thinking, message, tool-call tokens) substituted, because Yuan’s m would clash with the design m and its $C_{a,i}$ with the cost $C_m(p)$; exact with one call per step and full history; in general add $h_i^+ - h_i^-$ (other calls’ output; compaction)
$\sum_{i=1}^N H_{a,i} = N H_{a,1} + \frac{N(N-1)}{2} \bar{g}$	our distillation (calc.)	no source writes the sum; “at each step, the prompt sent to the LLM includes the history of all previous steps” (Abhyankar, Qi & Zhang, 2026)	$g_i = \Phi(z_{a,i}) + o_{a,i} $, $\bar{g} = \sum_i (N-i) g_i / \frac{N(N-1)}{2}$ makes it exact; over a random step count $\mathbb{E}[\sum H] \approx \mathbb{E}[N] H_{a,1} + \frac{\bar{g}}{2} (\mathbb{E}[N]^2 + \text{Var}[N] - \mathbb{E}[N])$; crossover $N^* = 2 H_{a,1} /\bar{g} + 1$

A0 · Money, per success, and the goal

the derivation behind page 4 · verbatim / adapted / our distillation · calc. = our algebra

FORMULA (PAGE 4)	STATUS	THE SOURCE’S OWN FORMULA OR WORDS, IN ITS NOTATION	WHAT CHANGED, AND WHY
$c_m(p) = \sum_{i,j} \sum_{\kappa} n_{ij}^{\kappa} c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}}$	adapted	$c_m(p) = n_{\text{in}}(m, p) c_{\text{in}}(m) + n_{\text{out}}(m, p) c_{\text{out}}(m)$ (Eq. 13): input and output tokens times their prices; other components go into $C_m(p) = w^{\top} x_m(p)$, w the unit prices and $x_m(p)$ the quantities per attempt (App. D.1) (Erol et al., 2026)	input split into the billed classes (cache hit, 5-min and 1-h cache write, uncached); prices inside the sum because one step can mix models; environment usage added as App. D.1 allows; the class index is κ because x is Cost-of-Pass’s quantity. Closest written three-class form: (Xu et al., 2026)
$v(m, p) = C_m(p) / R_m(p)$	verbatim	“the expected number of attempts to obtain the first correct solution is $1 / R_m(p)$ ”, assuming independent trials (§2.2, Eq. 2) (Erol et al., 2026)	none; assumes unlimited independent retries and the verifier’s time and cost in every attempt; with at most K retries the ratio is still C/R (calc.); with cache-warm retries $C_m^{(1)}(p) + (1 / R_m(p) - 1) C_m^{(2+)}(p)$ (calc.)
$\mathbb{E}[T_{\text{attempt}}] / R_m(p)$	adapted	“alternative units per attempt (FLOPs, time, latency, energy) may matter more than dollar cost” (App. D.1) (Erol et al., 2026)	the same derivation in seconds; serial retries only (parallel votes cost “roughly the latency of a single vote”); correlated failures break independence
$\text{Pareto}_m(\cdot, \cdot)$ s. t. $R_m(p) \geq R_0$	our distillation	$V_p(\mathcal{M}) = \min_{m \in \mathcal{M}} v(m, p)$ (Eq. 3), with a human expert as fallback, $\min(V_p(\mathcal{M}), v(\text{expert}, p))$ (Eqs. 4–5) (Erol et al., 2026); “the new goal of jointly optimizing the two metrics” (arXiv v1) (Kapoor et al., 2025); budgeted dual: maximise quality with expected cost within a budget (Chen, Zaharia & Zou, 2024)	min $\rightarrow$ Pareto set, because seconds and dollars are not one scalar; the human is the reference point; the floor R_0 follows App. C.8’s remedy of leaving unreliable systems off the frontier; over a task mix $\sum_p C_m(p) / \sum_p R_m(p)$, unlike Eq. 8’s mean of V_p

A0 · Every symbol, its unit, and where it comes from

origin of each symbol: “as in” a source (its own symbol and meaning), “adapted from” a source (renamed or widened), or self-defined

SYMBOL	MEANING	UNIT	ORIGIN
i, N, j, J_i, a	step (i = 0: calls outside any step); steps; model call; calls in step i; agent	—; steps; —; calls; —	i as in Chen et al., 2026 §3 and Yuan et al., 2026 §II-C · a as in Yuan et al., §II-C · N, j, J_i self-defined (calls ≠ steps)
$D_i, E_i, T_{\text{attempt}}, T_{\text{saving}}, T_{\text{seq}}$	model time; non-model time of step i; wall-clock of one attempt; time hidden by concurrency; a separate serial run	s	D adapted from Chen et al., 2026 §3 (all calls of a step) · E self-defined (replaces their T_i) · T_attempt self-defined · T_saving adapted from Feng et al., 2026 App. B.2 (intervals widened) · T_seq as in Ye et al., 2026 §3.1.2
$\mathcal{S}, \mathcal{D}, \mathcal{M}, \mathcal{E}, \triangle$	summed length; length of a union; model-call and non-model intervals; defined as	s	adapted from Feng et al., 2026 App. B.2 (intervals widened)
$\ell_{ij}, \text{TTFT}_{ij}, \text{TPOT}_{ij}$	call latency; time to first token; time per output token	s; s; s/token	ℓ as in Li et al., 2026 §2.1 (ℓ_LLM) · TTFT as in Artificial Analysis, 2026, definitions · TPOT as in Chen et al., 2026 §5.1 · indices self-defined
$t^{\text{queue}}, t^{\text{net}}, t^{\text{prefill}}, t^{\text{first}}$	parts of TTFT: queueing, network, prefill of uncached input, first decode step	s	self-defined (A0 · 1/5)
$n^{\text{in}}, n^{\text{hit}}, n^{\text{w}}, n^{\text{w5m}}, n^{\text{w1h}}, n^{\text{unc}}, n^{\text{out}}$	tokens: all input (= hit + w + unc); cache read; cache write (= w5m + w1h); uncached input; output	tokens	n as in Erol et al., 2026 (n_in, n_out) · classes as billed by Anthropic, 2026b (usage fields) · hit as in Xu et al., 2026 App. A.2 · unc as in Huang et al., 2026 App. A.3 · w5m, w1h self-defined
$\kappa, K, \mu_{ij}, c_{\kappa}(\mu)$	billing class; the set {hit, w5m, w1h, unc, out}; serving model; price per token of class κ on μ	—; —; —; \$/token	c adapted from Erol et al., 2026 Eq. 13 (c_in, c_out) · κ, K, μ self-defined
$x_{\text{env}}, c_{\text{env}}, w, x_m(p)$	billed environment usage; its price; price vector; quantity vector per attempt	h; \$/h; \$/unit; units	as in Erol et al., 2026 App. D.1 · env subscript self-defined
$H_{a,i}, \Phi, z_{a,i}, o_{a,i}, g_i, \bar{g}, N^*$	context before step i of agent a; chat template; the step’s output; its tool result; tokens added; weighted mean; crossover	tokens; —; tokens; tokens; tokens; tokens; steps	H, Φ, o as in Yuan et al., 2026 Eqs. 1–3 · z self-defined (their (θ, m, u)) · g, $\bar{g}$, N* self-defined (calc.)
$m, p, c_m(p), C_m(p), R_m(p), v(m,p), R_0, \mathbb{E}, \text{Var}$	design (model + harness); task; dollars of one attempt; expected; success probability; dollars per success; success floor; expectation; variance	—; —; \$; \$; prob.; \$; prob.	as in Erol et al., 2026 §2.2, Eq. 2, App. B · R_0 self-defined · $\mathbb{E}$, Var standard

A0 · What a published number covers: map it onto the terms before comparing

the measurement boundary · a figure is comparable only with what it includes stated

KIND OF FIGURE	WHAT IT INCLUDES	WHAT IT LEAVES OUT
first-token time, output speed (Artificial Analysis, 2026)	one synthetic call: queueing, network and prefill lumped in TTFT; speed after the first chunk	the step, the tools, the environment
request response time (Zhu et al., 2026)	model generation plus tool execution	the person thinking between turns
stopwatch time per task (Abhyankar, Qi & Zhang, 2026)	the whole attempt, T_{attempt}	— (39 tasks, one run each)
time saved (Feng et al., 2026)	decode and function-execution intervals of the concurrent trace	queueing, prefill, harness gaps
cost with two classes (input, output) (Erol et al., 2026)	all input at the base price	cache reads (overstates cost) and cache writes (understates it)
cost with three classes (hit, miss, output) (Xu et al., 2026)	cache reads at their price	cache writes; environment usage
cache hit rate	denominators differ: cached / cacheable input (Artificial Analysis, 2026), / all input (Xu et al., 2026), per incoming call (Luo et al., 2026)	ours: $n^{\text{hit}}/n^{\text{in}}$
almost every published figure	per attempt	per success: divide by $R_m(p)$; lossless speed papers report no task success

A1 · Slowness, every cause — I. steps and II. calls per step

the complete table behind pages 5–6 · three kinds of agent: screenshot desktop agent, text web agent, coding agent with a prompt cache

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
I-1	The task is long	Hour-scale tasks take hundreds of steps	OSWorld 2.0, 108 long tasks: Claude Opus 4.7 with maximum thinking, one action per step, 500-step cap: 318 tool calls per task on average, i.e. 318 steps; with several actions per call, Claude Opus 4.8 takes 103 steps (481.8 calls) and GPT-5.5 95.2 steps (149.8 calls)	(XLANG Lab, 2026)	screenshot, text
I-2	One action per step	Every click is a whole step: one observation, one model call, one wait	OSWorld 2.0: with one action per step Claude Opus 4.7 needs 318 steps; with batched actions the same model needs 160.7 and Claude Opus 4.8 needs 103	(XLANG Lab, 2026)	screenshot, text
I-3	Navigation-only steps still call the model	Paging, scrolling and opening the next page need no thinking but cost a call each	Skim’s profile of three text web agents (Browser-Use, AgentOccam, WebVoyager; GPT-4o) on 151 WebVoyager live-site tasks: 66.7% of the steps in the median task are pure navigation	(Wong et al., 2026)	text
I-4	Idling and dead loops	Element not found, repeated retries; steps rise without progress	OSWorld-Human’s records of the GTA1 harness (o3 plans and judges, GTA1-7B locates elements) on 39 OSWorld desktop tasks: in failed tasks that exceeded 50 steps, 66% of steps were wasted repeats; one element-locating loop repeated the same step 18 times — 27 minutes and \$8.47 at list price without caching	(Abhyankar, Qi & Zhang, 2026)	screenshot
I-5	Acting on a stale screen	The screenshot is taken before the page changes; the agent clicks on the old layout and must repair	OSWorld 2.0 records “stale interface state” as a failure mode; no share reported	(XLANG Lab, 2026)	screenshot
II-1	Multi-call harness	One step = plan + judge + reflect, several calls in series	OSWorld-Human, two harnesses: GTA1 runs 4 parallel planners per step, retries up to 3 rounds, then one judging call picks — 4–12 planning calls per judging call; Agent S2 adds a reflection call per step, so planning is 53% and reflection 34% of task time	(Abhyankar, Qi & Zhang, 2026)	screenshot
II-2	Sampling for accuracy	5–20 candidates per step, then pick one	Same model and harness (gpt-oss-120b, ReAct) on 165 WebArena-Lite tasks: 1 → 10 candidates per step raises success from 38.8% to 43.2% and tokens per task from 96K to 920K — 9.6× the tokens for 4.4 points	(Lee et al., 2026)	text
II-3	Coding agents call repeatedly within a turn	The model is called back and forth inside one user turn	GitHub Copilot coding agent production telemetry (first week of June 2026, 13.5 million sessions): 6.6 model calls per user turn on average	(Liu et al., 2026)	coding

A1 · Slowness, every cause — III. each call is slow

the complete table behind pages 7–8

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
III-1	Queueing	The request waits at the provider; unbilled, but counted as model time	Client-side measurement of 15 models across 5 providers: requests of the same length differ in latency by up to 69× depending on when they are sent	(Bian et al., 2025)	all
III-2	Reading (the prompt is re-processed every call)	Every call re-reads the whole history; more screenshots, slower	OSWorld-Human, multi-call harnesses: later steps up to 3× slower because the prompt at step i holds the $i - 1$ earlier screenshots — “dominated by prefill”. One screenshot is 1,000–1,800 tokens. AgentSysBench’s WebArena agent (Kimi-K2.6): observation switched from a single format to accessibility tree + HTML + screenshot → input 4.8×, model share of time 46.9% → 61.6%. Under load the cache is evicted and re-read: SWE-Agent / OpenHands running GLM-4.6 on 8 H100s for SWE-bench Lite, request latency up to 7.14× as concurrency rises	(Abhyankar, Qi & Zhang, 2026; Anthropic, 2026a; Chang et al., 2026; Kang et al., 2026)	screenshot; text with rich observations; coding under load
III-3	Writing (one token at a time)	Token-by-token generation, strictly serial	Locally served ReAct agents (Qwen3.6-27B / Gemma4-31B, vLLM, 2 H100s) on five benchmarks: with the cache warm, generation is 91–98.6% of model time. Windows desktop agent UFO2 (GPT-4o / o1 API): about 10 s per model call, the largest item per step in every configuration	(Yuan et al., 2026; Zhang et al., 2026)	coding; single-call screenshot
III-4	Thinking modes and larger models	Thinking writes more tokens; larger models are slower per token	OSWorld 2.0, same 108 tasks: Claude Opus 4.8 writes 224K output tokens per task, GPT-5.5 37K. Holistic Agent Leaderboard, 9 benchmarks, 21,730 runs: raising reasoning effort lowered accuracy in 21 of 36 pairs	(XLANG Lab, 2026; Kapoor et al., 2026)	all

A1 · Slowness, every cause — IV. environment, V. serial

the complete table behind page 9 (non-model time and concurrency)

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE	DOMINANT IN
IV-1	Producing the observation	Extracting the accessibility tree or detecting elements takes time	OSWorld desktop applications: 3–26 s to generate one accessibility tree; the screenshot itself is under 2% of task time. UFO2: OmniParser element detection adds about 1 s per step	(Abhyankar, Qi & Zhang, 2026; Zhang et al., 2026)	screenshot
IV-2	Browser execution and page load	After a click the agent waits for the page	151 WebVoyager live-site tasks: per step, median browser action 6.6 s vs model call 4.7 s. WebArena with GenericAgent + Claude 3.5 Sonnet: 7.6 s of a 12.2 s step in the browser (this figure was not re-verified)	(Wong et al., 2026; Le Sellier de Chezelles et al., 2025)	text
IV-3	Fixed sleeps	The agent cannot tell when the page is ready, so it sleeps a fixed number of seconds	OSWorld’s environment code sleeps 2 s after every action; OSWorld 2.0 prescribes 3 s, so 318 steps are about 16 minutes of pure waiting (calc.). AgentSysBench: a single-call GUI agent (ReAct, Kimi-K2.6 API) spends over 70% of its OSWorld execution time in the desktop sandbox	(xlang-ai, 2026; XLANG Lab, 2026; Chang et al., 2026)	screenshot (caused by the benchmark harness)
IV-4	Tool tails	Tests and builds take minutes	TraceLab, 4,265 Claude Code / Codex sessions from 43 developers (September 2025 – June 2026): tool calls over 1 minute are 4% of calls but 85% of tool time; a request averages 4.3 minutes — tools 2.5 (59.8%), model 1.7 (41.0%)	(Zhu et al., 2026)	coding
V	Serial structure	The four stages never overlap, so task time is the sum of every term, not the largest one; fixing one term saves only its share	Concurrency within a turn is 1.15 in Copilot telemetry. Separately, 80–92% of a coding session’s elapsed time is the human thinking between turns — strip it before reading any total	(Liu et al., 2026; Zhu et al., 2026)	all

A2 · Where the arrows land, with every measurement condition

the complete table behind page 10

	SCREENSHOT DESKTOP AGENT, MULTI-CALL HARNESS	TEXT WEB AGENT	CODING AGENT WITH A PROMPT CACHE
heaviest term	Reading × calls per step × steps. OSWorld-Human timed two multi-call harnesses (GTA1, Agent S2) step by step on 39 OSWorld desktop tasks: 87–97% of task time in planning, judging and reflection calls; screenshots and actions together under 3.5% (Abhyankar, Qi & Zhang, 2026)	Browser execution plus waiting. Skim measured three text web agents (GPT-4o) on 151 WebVoyager live-site tasks: per step, median browser action 6.6 s and model call 4.7 s (Wong et al., 2026)	Writing, or tool tails, depending on the tools. With the context cache on, token-by-token generation is 91–98.6% of model time (locally served models) (Yuan et al., 2026); in Claude Code / Codex records a request spends 59.8% in tools and 41.0% in the model (Zhu et al., 2026); in GitHub Copilot, whose tools are light, agent time is 13.7% model and 2% tools (Liu et al., 2026)
second	Writing: with thinking on, more reasoning tokens per step, and generation is one token at a time	Reading, once the observation grows: AgentSysBench switched a WebArena agent from one observation format to accessibility tree + HTML + screenshot — input 4.8×, model share 46.9% → 61.6% (Chang et al., 2026); the Browser-Use harness spends 73% of its latency in model calls (Winston et al., 2026)	Reading, when load evicts the cache and the whole context is re-read: ThunderAgent measured request latency up to 7.14× under cache thrash (Kang et al., 2026)
negligible	Producing the observation and executing the action: under 3.5% together (Abhyankar, Qi & Zhang, 2026)	Nothing: model and browser are close; which is larger depends on observation size and model speed	Navigation-only steps: a coding agent has no pages to turn
counter-example	A single-call agent is different: AgentSysBench’s one-call GUI agent (Kimi-K2.6) spends over 70% of its OSWorld time in the desktop sandbox (Chang et al., 2026), because the harness sleeps after every action — 2 s in OSWorld’s code (xlang-ai, 2026), 3 s in OSWorld 2.0 (XLANG Lab, 2026)	Browser-Use: 73% of latency in model calls, not the browser (Winston et al., 2026)	Two production records point opposite ways: heavy tools in Claude Code / Codex (tools 59.8% > model 41.0%) (Zhu et al., 2026); light tools in Copilot (model 13.7% > tools 2%) (Liu et al., 2026). The difference is the kind of tool (tests and builds vs file reads), not a contradiction
in one line	Slow in reading: every step re-reads the screenshot history, more as the task goes on	Slow in the environment (browser and page waits); once the observation grows, slow in reading again	Reading is cached away; slow in writing and in the tools

A3 · Cost, every cause — 1. tokens read and 2. tokens written

the complete table behind pages 8 and 11 · prices are list prices from the vendors’ own pages, September 2026

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
1-1	History re-sent	Step i reads everything from the $i - 1$ earlier steps; total reading $\approx N^2$	“Cost grows quadratically with the number of steps” — OSWorld-Human, real runs	(Abhyankar, Qi & Zhang, 2026)
1-2	Large screenshots	1,000–1,800 tokens per image; 100 images fill a 200K context	Anthropic’s engineering guidance for computer and browser use	(Anthropic, 2026a)
1-3	Rich observation formats	Accessibility tree + HTML + screenshot together: 4.8× the input	AgentSysBench, WebArena agent, Kimi-K2.6	(Chang et al., 2026)
1-4	Sampling	Several candidates per step, mostly read tokens	gpt-oss-120b + ReAct on 165 WebArena-Lite tasks: 1 → 10 candidates per step, 96K → 920K tokens per task	(Lee et al., 2026)
1-5	The cache is not a cure	However high the hit rate, a long history keeps reading dominant; a miss re-reads everything	TraceLab, 4,265 Claude Code / Codex sessions: prefix-cache hit rate 95.7%, prefix tokens still 59.5% of list-price cost; on a miss the re-read is 3.8× the genuinely new content	(Zhu et al., 2026)
2-1	Thinking modes	Thinking tokens are billed as output; when they cannot be switched off they are a fixed overhead	OSWorld 2.0, same tasks: Claude Opus 4.8 224K output tokens per task, GPT-5.5 37K; Claude Opus 5.5: “thinking cannot be disabled and is billed as output”	(XLANG Lab, 2026; Anthropic, 2026b)
2-2	But writing is not always the larger bill	In an uncached screenshot agent, output was 31% of the total	OSWorld-Human, GTA1: the paper’s \$2.43 counts output only; Table 3’s input and output give \$7.87	(Abhyankar, Qi & Zhang, 2026)

A3 · Cost, every cause — 3. calls, 4. failures, 5. unit price

the complete table behind page 11 (calls, failures, unit price)

ID	CAUSE	MECHANISM	EVIDENCE, WITH THE CONDITIONS OF THE MEASUREMENT	SOURCE
3	Number of calls	$N \cdot J_i$ scales both token bills	GTA1: 4 parallel planners per step, up to 3 retry rounds, one judging call → 4–12 planning calls per judging call, i.e. 5–13× the calls of a single-call harness (calc.). Agentic test-time scaling: 5–20 candidates per step multiply the calls by the candidate count. The same extra calls cost time and money at once	(Abhyankar, Qi & Zhang, 2026; Lee et al., 2026)
4-1	Failed attempts are billed	Cost per success = cost per attempt ÷ success rate	OSWorld 2.0: the best agent (Claude Opus 4.8) costs about \$72.4 per attempt at 20.6% completion → about \$351 per success (calc.). An accounting conversion, not the real price of retrying one task until it succeeds	(XLANG Lab, 2026)
4-2	Idle steps are billed in full	Every step of a dead loop is a full read and write	OSWorld-Human, GTA1 on 39 OSWorld tasks: one element-locating loop repeated the same step 18 times — 27 minutes, \$8.47 at list price without caching; in failed tasks over 50 steps, 66% of steps were such repeats	(Abhyankar, Qi & Zhang, 2026)
4-3	Evaluation itself is too expensive to repeat	Single runs, no repeats	Holistic Agent Leaderboard: 9 benchmarks, 21,730 runs, one run per configuration, about \$40,000 in total; Claude Opus 4.1 not run on Online-Mind2Web because the estimate was \$20,000	(Kapoor et al., 2026)
5-1	Model tier	Claude Opus 5.5 \$4 / \$20 vs GPT-6 Luna \$0.10 / \$0.50 per million tokens read / written	Vendor price pages, 27 September 2026	(Anthropic, 2026b; OpenAI, 2026)
5-2	Fast mode	2× on both vendors; switching speed tiers invalidates the cache	Anthropic: Claude Opus 5.5 \$8 / \$40, up to 2.5× output speed (vendor-stated), first-token wait unchanged. OpenAI: 2× on all listed models; “up to 2.5×” stated only for GPT-5.6 Sol	(Anthropic, 2026b; OpenAI, 2026)
5-3	Cache write and read	Latest model of each vendor, per million tokens, normal read → cached read (ratios calc.): DeepSeek-V4.1-Flash \$0.30 → \$0.006 (0.02×; peak rate, both halve off-peak) Claude Opus 5.5 \$4 → \$0.20 (0.05×; other Claude models 0.1×, Fable 5.1 0.025×; cache writes 1.25× for 5 min, 2× for 1 hour) GPT-6 Sol \$2 → \$0.20 and GPT-6 Luna \$0.10 → \$0.01 (0.1×; cache writes 1.25×) Gemini 3.8 Flash \$0.75 → \$0.075 (0.1×; plus \$0.50 per million tokens per hour of storage; introductory to 31 Dec 2026) Grok 4.7 \$2 → \$0.50 (0.25×)	Vendor price pages, read 28 September 2026	(DeepSeek, 2026; Anthropic, 2026b; OpenAI, 2026; Google, 2026; xAI, 2026)
5-4	Long-context surcharge	OpenAI: 2× the read price above 272K tokens	Vendor price page	(OpenAI, 2026)

A3 · Cost — 6. the environment machine, and the four conventions

the complete tables behind page 12 · vendor price pages read 2026-09-27 · per-task amounts and shares are our own arithmetic (calc.)

ENVIRONMENT	PRICE (VENDOR PAGE)	1-HOUR TASK	15-MIN TASK	SHARE OF A \$7.87 API BILL	SHARE OF \$72.4
Browser Use cloud browser	\$0.02 per browser-hour, billed by the minute, 1-minute minimum; traffic extra (residential proxy \$5/GB, direct \$0.20/GB); sessions capped at 240 minutes	\$0.020	\$0.005	0.25%	0.03%
AWS t3.medium (2 vCPU / 4 GiB, Linux, us-east-1)	\$0.0418 per hour on the T3 product page; another AWS page says \$0.0416 (0.5% apart)	\$0.042	\$0.010	0.5%	0.06%
Browserbase cloud browser	\$20 per month for 100 hours, then \$0.12 per hour; \$99 per month for 500 hours, then \$0.10; billed by the minute	\$0.12	\$0.03	1.5%	0.17%
E2B / Daytona sandbox (2 vCPU / 4 GiB)	E2B \$0.000014 per vCPU-second + \$0.0000045 per GiB-second; Daytona \$0.0504 per vCPU-hour + \$0.0162 per GiB-hour — both \$0.1656 per hour	\$0.166	\$0.041	2.1%	0.23%
Modal sandbox (1 physical core = 2 vCPU, 4 GiB)	\$0.00003942 per core-second + \$0.00000667 per GiB-second → \$0.238 per hour; billed on the larger of requested and used	\$0.238	\$0.060	3.0%	0.33%
AWS t3.2xlarge (8 vCPU / 32 GiB, OSWorld 2.0's default instance)	\$0.3341 per hour	\$0.334	\$0.084	4.2%	0.46%
OpenAI hosted container (Hosted Shell / Code Interpreter, 4 GB)	\$0.12 per 20-minute session → \$0.36 per hour; 1 GB \$0.03, 16 GB \$0.48, 64 GB \$1.92 per 20 minutes	\$0.36	\$0.12	4.6%	0.50%

The four accounting conventions (page 12), in one line each: output only vs all tokens — 3.2× (OSWorld-Human, \$2.43 vs \$7.87); per attempt vs per success — 4.9× (OSWorld 2.0, \$72.4 ÷ 20.6%); uncached vs cached — 4–50× on the read price by vendor (calc.), far less on the bill (TraceLab: 95.7% hits, prefix still 59.5% of cost); standard vs fast mode — 2× (both vendors), buying only writing speed.

A4 · Each cause’s effect on time and on money

the complete table behind pages 13–14

CAUSE	EFFECT ON TIME	EFFECT ON MONEY	RELATIONSHIP
Many steps (N)	Roughly linear: one more step is one more call and one more wait (later steps somewhat slower, up to 3×)	Write tokens linear in N ; read tokens quadratic, because every step re-reads the whole history	same source — double the steps: about 2× the time, about 4× the read bill (uncached)
Many calls per step (J_i)	linear	linear	same source
Reading a lot (history, screenshots, rich observations)	Reading time, longer every step	Read-token bill	same source — the same tokens cost time and money
Writing a lot (thinking)	Generation time, roughly 12 ms per token	Write-token bill at 5× the read price	same source , and the dearest time: writing is the slowest and the most expensive token
Cache hits	Saves reading time	Read price 0.02–0.25× of a normal read, by vendor (A3)	same source , but fragile: editing the history or switching to fast mode invalidates it
Queueing	slow	free	slow but not expensive ; escaping it means a fast or priority tier at 2×
Environment waits (page loads, fixed sleeps)	slow	No API cost; machine time by the hour — CPU sandboxes \$0.02–0.36 per hour, usually under 5% of the API bill	slow but not expensive
Tool tails (tests, builds)	slow	No API cost	slow but not expensive
Failures, idling, retries	slow	expensive	same source , and it turns “per attempt” into “per success”
Fast mode	Writing up to 2.5× faster (vendor-stated); reading unchanged	2× the price	money for time
Bigger models, more thinking	slower	dearer	money and time for accuracy , convex: marginal tokens per point 101K → 575K on one model; 6× / 9× / 9.6× tokens for a few points; more thinking not always more accurate (Lee et al., 2026; XLANG Lab, 2026; Kapoor et al., 2026)
Smaller models	faster	cheaper	but if accuracy falls, steps and retries rise — possibly slower and dearer again (inference, no direct evidence)

A5 · The gaps in the evidence, and what each one blocks

the complete table behind page 14

GAP	WHAT EXISTS TODAY	CONSEQUENCE FOR WHAT CAN BE CLAIMED
Read / write time split and cache hit rate for a frontier API model	No measurement reports both; the only read / write split is on locally served 27–31B models (Yuan et al., 2026); screenshot-agent measurements do not report cache state	How much of “87–97% in the model” is reading, how much writing, and what remains once the cache is on — unknown
How much slower an agent is than a person	None of 17 benchmarks measures agent and human elapsed time on the same tasks; the closest is AXIS’s user study, in which a UI agent was 1.69× slower than manual work on easy tasks (small sample) (Lu et al., 2025)	“X times slower than a person” cannot go on a slide
Environment dollars	Benchmarks report instance types; AgentSysBench reports shares; products publish nothing. Computable from instance type × duration × list price: CPU environments usually 0.3–5% of the API bill (page 12)	Every benchmark cost is an API bill; the omission is small for CPU environments, not for GPU sandboxes or long idle sessions
Whether OSWorld-Human’s “action” includes the 2-s sleep after each step	Not stated (xlang-ai, 2026); if it does, “actions under 2%” implies more than 100 s per step (calc.)	“Environment under 3.5%” may understate the waiting
The rebound from a smaller model	No direct evidence on how many extra steps and retries a drop in accuracy causes	The last row of the relationship table stays marked “inference”
How much slowness hurts accuracy	Only OSWorld 2.0 records the stale-state failure mode, without its share of failures (XLANG Lab, 2026)	“It exists” can be said; “how large” cannot

PART 2

Acceleration, term by term

What published work has done to each term of the Part 1 formulas, and what each change costs elsewhere.

We classify; we do not propose methods. Each work sits on the term it changes, next to the terms that grow in exchange and one measured effect with its conditions. Every number was re-read in its source on 28 September 2026; calc. marks our arithmetic, vendor a vendor’s own statement. Full references at the end of the part.

IN THIS PART

02	The map: which term each family of work changes
03–04	Fewer model calls, fewer steps
05–07	A faster call: queueing, prefill, decoding
08	Less time outside the model
09–10	Overlap: parallel calls, and speculation with its safety
11–13	The growing prompt, the bill by class, a cheaper model per call
14	Success and cost per success, set-up included
15–20	References · Appendix B0–B10

The map: each family of work targets one term or a pair, and most grow another

from Part 1’s formulas → here: which term each family of work changes → to: one page per term

Each work sits on the term it changes: it shrinks a term, removes a structure, or adds overlap T_{saving} ; most also grow another term. Speed-up = baseline time ÷ new time on the same tasks; each row says what was timed.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \left(t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \text{TPOT}_{ij} \right) + E_i \right] - T_{\text{saving}}$$

MONEY

$$v(m, p) = \mathbb{E} \left[\sum_i \sum_j \sum_{\kappa} n_{ij}^{\kappa} c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}} \right] \div R_m(p)$$

TERM (PART 1)	WHAT THE WORK DOES TO IT	FAMILY OF WORK	PAGE
J_i	→ 0 on steps run as code or replayed from a recording	compile or replay the loop	03
N	fewer steps: more work per call, reusable skills	bigger actions, skills	04
$t^{\text{queue}}, t^{\text{prefill}}$	shorter on self-hosted serving: whole programs scheduled, cache kept across tool waits	agent-aware serving	05
$ o_{a,i} \rightarrow t^{\text{prefill}}$	less of each page shown to the model	observation reduction	06
$n^{\text{out}}, \text{TPOT}$	less thinking; faster decoding, at a price	decoding	07
E_i	fewer or cheaper environment actions	environment	08
T_{saving}	added: independent calls run at once, or the next step guessed and run early	parallel calls; speculation	09–10
$\bar{g}, N^2$	a flatter slope; or the square removed	context management	11
n^{κ}	input moved from uncached to cache hit	caching	12
$c_{\kappa}(\mu_{ij})$	a cheaper model or tier for most calls	routing, small models, tiers	13
$R_m(p)$, set-up	higher success; a one-time set-up spread over the tasks	cost per success	14

Compiling or replaying the loop removes most model calls; where the actions still run, time falls less

term: $J_i \rightarrow 0$ on steps run as code or replayed · E_i unchanged on replay; JIT-Planner’s cached site tools also shorten it · a set-up cost sits outside the formula (page ...

Code or a recorded trajectory takes the model out of most steps, so $D_i \rightarrow 0$ there; the actions in E_i still run unless the code replaces them too. Against Synapse, a planning agent, replay cut median tokens by 96.5% but median time by 59.6% (calc., EchoPath).

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \rightarrow 0 \left(t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \text{TPOT}_{ij} \right) + E_i \right] - T_{\text{saving}}$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
JIT-Planner (Winston et al., 2026)	Parallel calls draft code plans over cached, checked site tools; the cheapest valid one runs	Task time 150.1 → 15.4 s (9.7×), success 61 → 90% — 37 tasks on 5 web apps (18 author-written), 3 runs, GPT-4.1, vs Browser-Use, same model	Set-up of 25–90 + 25–45 min per app, excluded
ActionEngine (Zhong et al., 2026)	An offline state graph; one call writes a program, graph search compiles it	Model calls 10.2 → 1.8, task time 237 → 118 s, success 66 → 95% — 106 WebArena Reddit tasks	Baseline: another agent and model; crawl cost unreported
AutoDroid-V2 (Wen et al., 2025)	A local model writes one script per task, not one call per step	Model time 669.2 → 46.3 s per task (screen actions excluded), success 43.9 → 54.4% — 158 DroidTask tasks, same 8B model	\$82.42 of GPT-4o calls per app, offline (calc.)
EchoPath (Zhao et al., 2026)	Validated trajectories kept as callable memories and replayed	Median tokens 586,386 → 20,370, median time 315.7 → 127.5 s, success 91.8 → 91.2% — OSWorld-Verified, 159 replayable tasks, Codex GPT-5.5, vs Synapse	First pass ≈ 572k tokens and ≈ 4.5 min per task, not counted

Vendor tools replay recorded steps and call a model only to repair a failure; none publishes a repair rate with a defined sample (B1).

Fewer steps: more work per call cut steps by 38–46%; skills cut fewer and cost prompt tokens

term: $N \downarrow$ · actions per step grow with batching, fall with API calls (page 8) · skills add module calls ($J_i \uparrow$) and lengthen the prompt $|H_{\{a,i\}}|$

One call that issues an API call or a batch of actions cut N by 38–46% (calc.); the actions still run, and only **AXIS timed the task.** Skills cut 11–21% of steps (calc.) and raise success; a plain agent given 15 steps instead of 10 used 31% fewer tokens (calc.) and did better.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \downarrow \left[\sum_{j=1}^{J_i} \uparrow \left(t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} \uparrow + n_{ij}^{\text{out}} \text{ TPOT}_{ij} \right) + E_i \right] - T_{\text{saving}}$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
AXIS (Lu et al., 2025a)	Application API calls instead of UI sequences	Steps 3.2 → 2.0, task time 59.5 → 29.9 s, success 52 → 84% — 50 Word tasks, GPT-4o, vs the UFO agent	One application; the API must exist
OSWorld 2.0, batched (XLANG Lab, 2026)	One call emits several actions	Steps 190.5 → 103, success 18.5 → 20.6%, ~\$76.1 → ~\$72.4 per task — 108 tasks, Claude Opus 4.8, one run	Tool calls 190.5 → 481.8; no wall-clock
ASI (Wang et al., 2025b)	Verified Python skills induced from the agent’s own successes	Steps 5.6 → 5.0 (~10.7%, calc.), success 32.7 → 40.4% — 812 WebArena tasks, Claude 3.5 Sonnet	Induction calls not counted; no time or \$
WALT (Prabhu et al., 2026)	Site tools learned by exploration	Steps 8.9 → 7.0, success 57.5 → 64.1% — 234 VisualWebArena tasks, GPT-5 planner, GPT-5-mini executor	Exploration cost not quantified
Plain agent, 15 steps (Hajimiri et al., 2026)	No skill or memory module; a pruned page tree	Plain 44.78% at 73.6K tokens per task vs ASI 41.02% at 107.3K — 655 WebArena tasks (calc.), Gemini 3 Flash, 3 runs	Skills add tokens and calls

Headroom: grouping actions by screenshot needs 1.35× (Chrome) to 2.93× (Calc) fewer steps in human reference runs (calc.) (Abhyankar, Qi & Zhang, 2026).

Serving the agent as one program cuts queueing and re-reading, on self-hosted models only

terms: t^{queue} and $t^{prefill}$ ↓ under load · self-hosted serving only; an API user cannot apply these

Schedulers that treat a whole agent run as the unit, and keep its cache through tool waits, give 1.1–15× better delay or throughput than vLLM-based baselines. All are measured under load on self-hosted models; none reports task success.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \left(t_{ij}^{\text{queue}} \downarrow + t_{ij}^{\text{prefill}} \downarrow + n_{ij}^{\text{out}} \text{TPOT}_{ij} \right) + E_i \right] - T_{\text{saving}}$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
Agentix (Luo et al., 2026)	Program-level preemptive scheduling; cache-aware routing	Program throughput up to 15× vLLM and 2–5× vLLM with prefix caching, at equal latency — Llama-3.1 8B/70B and Falcon-180B on A100s	No task quality: scheduling only
Continuum (Li et al., 2026a)	Keeps a run’s KV cache for a predicted tool-wait time	Job delay 1.12–3.66× lower — replayed SWE-bench, BFCL and OpenHands traces; Llama-3.1 8B/70B, Gemma-3 12B	GPU memory held during waits; replay, no quality metric
InferCept (Abhyankar et al., 2024)	During a tool call, keeps, swaps or recomputes the cache, whichever wastes least	1.6–2× the load of vLLM at similar per-token latency (1.25× for 13B on one GPU) — six tool-augmented workloads	Baseline spent 37–40% of forward time recomputing
ThunderAgent (Kang et al., 2026)	The program and its tool sandboxes are the scheduling unit	Steps per minute 1.24–3.58× vLLM (calc. from its figure) — GLM-4.6 and Qwen-3 on 8×H100	Throughput, not task time

Showing the model less of the page cuts prefill; a reader call can cost more time than it saves

term: $|o_{\{a,i\}}| \downarrow \rightarrow$ the actor's t^{prefill} and $n^{\text{unc}} \downarrow \cdot J_i + 1$ when a model does the reducing; that reader reads the full page · success can fall

Pruning the page before the model reads it shortens the actor’s prompt at that step, and later prompts only where old pages stay in the history; a separate reader call adds a model call per step. A pruning program without a model halved step time and kept 84% of the successes.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \uparrow (t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} \downarrow + n_{ij}^{\text{out}} \text{TPOT}_{ij}) + E_i \right] - T_{\text{saving}}$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
FocusAgent (Kerboua et al., 2026)	A small model picks the relevant page-tree lines for the actor	Input cost −19%, success 53.6 → 51.5% — 330 WorkArena L1 episodes; model time per step 2.5 → 10.1 s on 33 tasks, one seed — GPT-4.1 actor, GPT-4.1-mini reader	One reader call per step; the time excludes the browser
Evolved pruning program (Enomoto et al., 2026)	An offline-evolved program keeps ~20% of the HTML; no model at run time	Step time 65.7 → 30.2 s (2.2×), 84% of successes kept — 33 WorkArena L1 tasks, Qwen3.5-122B, 2 runs	Success falls; timed per step
Aguvis (Xu et al., 2025)	A screenshot instead of HTML, read by a self-hosted 72B model	Input tokens per step 1,196 vs ~4,000 (−70%), success 22.1 → 27.1% — 104 Mind2Web-Live tasks, vs GPT-4o on HTML	The model changes too
AgentOccam (counter) (Yang et al., 2025)	Simplified page and pruned history, aimed at accuracy	Success 16.5 → 43.1%, observation tokens per step +33%, steps +45% (calc.) — 812 WebArena tasks, GPT-4-Turbo	Prefill grows

Less thinking and faster decoding shorten the writing; neither touches the environment

terms: $n^{\text{out}} \downarrow$ (thinking included) · TPOT $\downarrow$ at a higher price per token

Choosing the reasoning effort per step cut reasoning tokens by 45% at equal success; fast tiers are stated by their vendors to be up to 2.5× faster for twice the price. Both shorten D_i only.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \uparrow (t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \downarrow \text{TPOT}_{ij} \downarrow) + E_i \right] - T_{\text{saving}}$$

MONEY

$$v(m, p) = \mathbb{E} \left[\sum_i \sum_j \sum_{\kappa} n_{ij}^{\kappa} \left(c_{\kappa}(\mu_{ij}) \uparrow + x_{\text{env}} c_{\text{env}} \right) \right] \div R_m(p)$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
ARES (Yang et al., 2026a)	A 1.7B router picks the reasoning effort for each step	Reasoning tokens per task 21,424 → 11,723 (−45%), success 45.0 → 46.5%; low effort throughout −7.6 points (calc.) — WebArena, ~129 tasks (calc.; count not printed, B9), gpt-oss-20b	A router call per step, cost not reported; no wall-clock
Overthinking (Cuadron et al., 2025)	Two low-effort runs; keep the one that overthinks least	\$1,400 → \$800 for the whole run, solved 29.1 → 27.3% — SWE-bench Verified (count not printed, B9), o1	Two attempts per task
GUI-G1 (Zhou et al., 2025)	RL-trained grounding without a thinking section	Output tokens 107–114 → 37–39 per example, ScreenSpot 87.5 → 90.3% — InfiGUI-R1-3B (accuracy from its own paper) vs GUI-G1-3B, two differently trained models	Grounding only; no timing
ToolSpec (Xia et al., 2026)	Schema-aware drafts of tool-call tokens, verified losslessly	Tool-call generation 3.5–4.2× faster — API-Bank, ToolAlpaca, BFCLv2; batch 1; 3–14B open models	Decoding speed, not task time
Fast tiers (vendor) (Anthropic, 2026b; OpenAI, 2026)	The same model, served faster	Anthropic: up to 2.5× output tokens per second; the gain is in output speed, not first-token time. OpenAI: up to 2.5× faster, ‘speed’ undefined	Price 2×; switching speed invalidates Anthropic’s message cache

Time outside the model: in one browser-agent study it exceeded model time, and few works shrink it

term: $E_i \downarrow$ – fewer environment actions, cached tool results · x_{env} priced by the hour

On 151 WebVoyager tasks run by three GPT-4o agents on live sites, the median step spent 6.6 s in the browser and 4.7 s in the model (medians, not additive) (Wong et al., 2026); Browser-Use on five web apps spent 73% of its time in model calls (Winston et al., 2026). The works that shrink E_i skip the browser or cache tool results; batching actions moves the other way.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \left(t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \text{TPOT}_{ij} \right) + E_i \downarrow \right] - T_{\text{saving}}$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
Skim (Wong et al., 2026)	Fetches a synthesized URL over HTTP instead of driving the browser; a verifier falls back to the agent	Median cost 1.9× lower, latency −33.4%, accuracy kept — WebVoyager and WebShop, “300+” tasks (count not stated; condition (iii) pending, B9), GPT-4o agents	Read-only tasks; site profiling not counted
ToolCaching (Zhai et al., 2026)	Caches tool results, with learned admission	Latency 16.2 → 10.7 s vs no cache at a 0.514 hit ratio — 500 Movie Recommendation queries, LLMCompiler, DeepSeek V3	Accuracy not reported
AXIS (Lu et al., 2025a)	API calls replace clicks	UI actions 103 → 48 over 50 Word tasks	The API must exist
OSWorld 2.0 (counter) (XLANG Lab, 2026)	More actions per call	Tool calls ×2.5 (calc.); a 3-s pause follows each action, whether after each batched call is not stated; no wall-clock	N falls, actions per step grow

Billed environment usage $x_{\text{env}}c_{\text{env}}$: a cloud browser costs \$0.10–0.12 per hour above the plan allowance (vendor price) (Browserbase, 2026b).

Overlap (1): running independent calls and tools together saved 31–73% of task time; one workload got slower

term: T_{saving} raised where it is small today · calls and tokens fall with fewer turns (LLMCompiler, W&D) or rise with more workers (ParaGUI)

Running independent calls or tools together creates T_{saving} ; the longest dependent chain sets the limit. Speed-up $\leq (T_{\text{LLM}} + T_{\text{tool}}) / \max(T_{\text{LLM}}, T_{\text{cp}})$: decoding, tool and critical-path tool time (Feng et al., 2026); at most 2× when every tool call waits for the one before ($T_{\text{cp}} = T_{\text{tool}}$; calc.).

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \uparrow \text{ or } \downarrow (t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \text{ TPOT}_{ij}) + E_i \right] - T_{\text{saving}} \uparrow$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
LLMCompiler (Kim et al., 2024)	A planner builds a dependency graph; independent calls run at once	Task time 20.47 → 5.47 s (3.74×), accuracy 72.47 → 77.13%, input tokens 20,000 → 2,800 – 500 Movie Recommendation questions, gpt-3.5-turbo	WebShop got slower: 5.98 → 10.48 s
W&D (Lin et al., 2026)	Three or more tool calls per turn	Time per task 1,522.6 → 904.2 s (–40.6%), \$102.5 → \$65.7 per 100 tasks, 66 → 68% – first 100 BrowseComp tasks, GPT-5, one run	Call counts forced; \$ include search
AsyncFC (Feng et al., 2026)	Tool calls return futures; the model keeps decoding until it needs a result	1.44× vs the plain agent (1.21× with parallel calls alone), resolved 47.6 → 44.3% – 300 SWE-bench Lite tasks, GPT-5.2, tool latency doubled	Resolution –3.3 points
ParaGUI (Yu et al., 2026)	A planner sends sub-tasks to up to 5 GUI workers on separate desktops	Critical-path steps 38.7 vs 75.9, success 46.4 vs 33.5%, vs Claude Sonnet 4.6 – 233 tasks; vs its own worker model 38.7 vs 36.7	Desktops × workers; no wall-clock

Overlap (2): guessing the next step saved 5–45% of task time, always paid in extra calls

term: T_saving added by guessing · calls, tokens and GPU hours grow · safe only for effects that can wait for a commit

A guesser (a cheaper model or setting, or the model itself) runs the predicted next action early: a right guess hides a wait, a wrong one is discarded but paid for. Steps with side effects need the rule on the right.

TIME

$$T_{\text{attempt}} \approx \sum_{i=0}^N \left[\sum_{j=1}^{J_i} \uparrow (t_{ij}^{\text{queue}} + t_{ij}^{\text{prefill}} + n_{ij}^{\text{out}} \text{TPOT}_{ij}) + E_i \right] - T_{\text{saving}} \uparrow$$

MONEY

$$v(m, p) = \mathbb{E} \left[\sum_i \sum_j \uparrow \sum_{\kappa} n_{ij}^{\kappa} \uparrow c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}} \right] \div R_m(p)$$

targeted

in exchange

unchanged

WORK	MEASURED EFFECT, AND ITS CONDITIONS	PAID IN
Speculative Actions (Ye et al., 2026)	Time −19.5%, guesses 54.7% right — chess, 5 × 30 steps, GPT-5 high effort guessed by GPT-5 low	Guess calls; ≤ 50% saving in its model
ISP (Hua et al., 2025)	Total time 182.70 → 105.42 s (−42.3%), \$0.2160 → \$0.2973 per plan — 117 OpenAGI tasks, GPT-4-turbo	Baseline spread 2.3× its mean
SMC (Liu et al., 2026c)	τ² Telecom, 2,285 tasks (calc.): 27.60 → 22.47 s (−18.6%), outcomes unchanged; AppWorld: 355.7 → 195.9 s (−44.9%; −40.4% already from one-step guessing), −2 of 168 — Qwen3.5-27B, 4B drafter	3 GPUs instead of 1
SPORK (Bai et al., 2026)	p50 task time 34.7 → 31.2 s (−10%), p95 131.9 → 108.1 s (−18%), exact match within 1 point — 165 GAIA tasks, Qwen3-32B, baseline already speculative-decoding	Probe calls, wasted tool runs
Ghost Tool Calls (Mohammadi et al., 2026a)	Naive speculation: p50 −5.4%, p99 11.56 → 14.22 s; tool calls seen by the provider 4.03 vs 1.03 per task — 30 author-built tasks × 3 seeds	Abandoned calls reveal intent

WHEN THE STEP HAS SIDE EFFECTS

Hold irreversible effects until commit; compensating them afterwards leaks

- Held until commit: 0 of 500 aborted sends leaked; compensation 400, checkpoint-and-replay 200, mislabelled effects 300 — one author-built workflow, 5 abort causes × 100 (Mohammadi et al., 2026b)
- Resuming from a checkpoint repeated or skipped an effect in 90 and 93 of 96 author-built workflows; a case study paid an invoice twice (Wu et al., 2026)
- Only ~37% of actions flagged destructive in advance were (Dihan et al., 2025)

The growing prompt: masking old observations halved the bill; only compiled plans are shown to remove the square

terms: $\bar{g} \downarrow$ (mask, trim, evict) or the N^2 structure removed (a plan instead of the history) · edits to old context turn some cache hits into misses or cache writes

Placeholders remove each old observation once it is 10 turns old: a smaller effective $\bar{g}$, a flatter slope of the N^2 term but the same shape. A compiled plan removes the square: the prompt holds the plan, not the history.

PROMPT

$$\sum_{i=1}^N |H_{a,i}| = N |H_{a,1}| + \frac{N(N-1)}{2} \text{ removed by a plan } \bar{g} \downarrow$$

MONEY

$$v(m,p) = \mathbb{E} \left[\sum_i \sum_j \sum_{\kappa} \left(n_{ij}^{\kappa} \text{ hit} \rightarrow \text{miss, write} \right) c_{\kappa}(\mu_{ij}) + x_{\text{env}} c_{\text{env}} \right] \div R_m(p)$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
Observation masking (Lindenbauer et al., 2025)	Observations older than 10 turns become a placeholder	Cost per task \$1.29 → \$0.61, solved 53.4 → 54.8% — 500 SWE-bench Verified tasks, Qwen3-Coder-480B; cost = tokens × list price	Model summaries instead lengthened runs by ~15%
AgentDiet (Xiao et al., 2026)	A cheaper model deletes useless or expired content two steps back	Input tokens −39.9 to −59.7%, cost −21.1 to −35.9% with the reducer, success −1 to +2 points — 200 SWE-bench Verified + 300 Multi-SWE-bench Flash tasks, Claude 4 Sonnet, Gemini 2.5 Pro, one run	A reducer call per step
TokenPilot eviction (Xu et al., 2026a)	Evicts stale segments every 3 turns	Cost \$4.22 → \$2.79 while cache-hit tokens fell 26.7M → 8.6M — PinchBench continuous sessions, GPT-5.4-mini	Fewer tokens, fewer of them cached
A plan, not the history (Zhong et al., 2026; Kim et al., 2024)	The prompt carries a compiled plan instead of every observation	Input tokens per task 62.3k → 8.1k (ActionEngine); 20,000 → 2,800 (LLMCompiler)	Only on compiled steps

The bill by class: caching a stable prefix cut agent bills by 41–80% against no caching; with trimming and eviction, by up to 87%

term: $n^{\text{unc}} \rightarrow n^{\text{hit}}$ (input read from the cache) · cache writes n^{w} grow · the price $c_{\kappa}(\mu)$ unchanged

A cache hit costs 2–25% of an uncached input token (by vendor), so moving input into the hit class cuts that input’s price by 75–98% (calc.) without changing the model. A change of speed, of tools or of old context turns hits back into writes and misses.

MONEY

$$v(m, p) = \mathbb{E} \left[\sum_i \sum_j \sum_{\kappa} \left(\overset{\text{targeted}}{n_{ij}^{\kappa} \text{ unc} \rightarrow \text{hit}} \overset{\text{in exchange}}{c_{\kappa}(\mu_{ij}) =} \overset{\text{unchanged}}{+ x_{\text{env}} c_{\text{env}}} \right) \right] \div R_m(p)$$

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
Don’t Break the Cache (Lumer et al., 2026)	Measures caching the stable prefix and leaving tool results out	Cost –41 to –80%, first-token time –6 to –31% vs a forced no-cache baseline — a web-search research agent on DeepResearch Bench, 40 sessions per condition, 4 models, a 10K-token system prompt	Caching the whole context made GPT-4o’s first token 8.8% slower
TokenPilot (Xu et al., 2026a)	Stable placeholders for changing fields, trimming, eviction	Cost \$8.31 → \$3.22 (123 PinchBench tasks, one per session), \$81.52 → \$10.58 (Claw-Eval, continuous); hit rate 38.7 → 79.2% — GPT-5.4-mini	Score change –2.6 to +2.1 points
TraceLab counterfactual (Zhu et al., 2026)	Keeps the cache alive through the user’s think time	At most –12.8% of the priced cost (paper; 15.8% on the live dashboard) — coding-agent traces	An estimate, not a measurement

LATEST MODEL (VENDOR PRICE PAGES, READ 28 SEP 2026)	CACHE HIT ÷ UNCACHED INPUT	CACHE WRITE
DeepSeek-V4.1-Flash	0.02×	not listed
Claude Opus 5.5	0.05×	1.25× (5 min), 2× (1 h)
GPT-6 Sol, GPT-6 Luna	0.1×	1.25×
Gemini 3.8 Flash	0.1×	storage \$0.50 per million tokens per hour
Grok 4.7	0.25×	not listed

A cheaper model for most calls cut dollars per task by 50–97%, usually for a few points of success

term: $c_k(\mu_{ij}) \downarrow$ – a smaller model or cheaper tier for most calls · N and J_i can grow · $R_m(p)$ often falls

Sending most steps to a small model, and only hard ones to a large one, cut dollars per task by 50–97% (calc.), usually for 1–4 points of success and sometimes with more steps. Tiers trade the price per token against queueing and speed.

MONEY

$$v(m, p) = \mathbb{E} \left[\sum_i \uparrow \sum_j \sum_{\kappa} n_{ij}^{\kappa} c_{\kappa}(\mu_{ij}) \downarrow + x_{\text{env}} c_{\text{env}} \right] \div R_m(p) \downarrow$$

targeted

in exchange

unchanged

WORK	WHAT IT CHANGES	MEASURED EFFECT, AND ITS CONDITIONS	WHAT GROWS, OR IS LEFT OUT
StepWise (Wei et al., 2026)	An 8B model by default; the large model when the small one is stuck	\$0.881 → \$0.224 per task (−74.6%), success 58.1 → 55.4%, time per request 6.4 → 4.1 s — OSWorld (count not printed, B9), EvoCUA-8B + Claude Sonnet 4.5 vs Sonnet 4.5 alone	Steps 25.4 → 26.2
WebRouter (Li et al., 2026b)	A router picks one of three models for each step	\$0.98 → \$0.12 per task, accuracy 86.1 → 82.3%, ~14% slower — 5 WebVoyager sites, ≥ 46 tasks each, vs GPT-4o	Steps 7.63 → 8.38
Fara-7B (Awadallah et al., 2025)	A 7B screenshot agent instead of a frontier computer-use model	\$0.913 → \$0.025 per task, success 70.9 → 73.5% — WebVoyager (count not printed, B9), 3 runs; but Online-Mind2Web 42.9 → 34.1%	7B priced at \$0.20 per million tokens, no caching
Agentic plan caching (Zhang et al., 2025)	Cached plan templates, adapted by a small model	Cost −50.31% on average, at 96.61% of the best accuracy — 5 workloads; latency −27.28% on one FinanceBench run	3.99 s to build each cache entry

Tier prices (vendor, read 28 Sep 2026): OpenAI Flex and Batch 0.5×, Fast 2×; Google Priority 1.8× (Preview); Microsoft Foundry priority 1.75–2.5× (calc.) (OpenAI, 2026; Google, 2026; Microsoft, 2026a).

Cost per success moves with the success rate, and a set-up cost must be spread over the tasks

terms: $R_m(p)$ in $v(m,p) = C_m(p) / R_m(p)$ · plus a one-time set-up cost that Part 1’s per-attempt formula leaves out

A dearer attempt can give a cheaper success: Beyond Browsing’s hybrid agent costs \$1.4 per attempt against \$1.2 for API calls alone, but \$3.60 against \$4.11 per success (calc.). Few papers report the ratio, so the figures here are our calculations. Set-up costs are left out of most headlines; SpeedRunner spreads them over its runs, EET prints them apart (B8).

PER TASK
SET-UP
SPREAD
adapted

$$v_n(m,p) = \frac{C_{\text{setup}}}{n} + \frac{C_m(p)}{R_m(p)}$$

C_{setup} : one-time cost of building the code, tools or memories; n : tasks it serves; the second term is Part 1’s $v(m,p)$, each task retried until it succeeds, so per task = per success. Against a baseline of cost per success v_{base} it pays off once $n > C_{\text{setup}} / (v_{\text{base}} - v(m,p))$, which needs $v(m,p) < v_{\text{base}}$ (calc.).
fixed + variable cost: Kapoor et al., 2025, §3 (in words); one large tool-making call spread over n cheap calls, $O(nc + C)$ with C, c the cost of one large- and one small-model call: Cai et al., 2024, Table 2 · per success: Erol et al., 2026, Eq. 2

WORK	COST PER SUCCESS (CALC. UNLESS STATED)	WHY IT MOVED
AXIS (Lu et al., 2025a)	\$0.77 → \$0.24	cheaper and more successful
W&D (Lin et al., 2026)	\$1.55 → \$0.97 per correct answer	cheaper per attempt
OSWorld 2.0, batched (XLANG Lab, 2026)	\$411 → \$351	Opus 4.8; success 18.5 → 20.6%
Beyond Browsing (Song et al., 2025)	browsing ≈ \$0.68 · API only ≈ \$4.11 · hybrid ≈ \$3.60	dearer attempt, cheaper success (hybrid vs API)
BATS (Liu et al., 2026a)	\$0.79 → \$4.47	success 12.6 → 24.6%, cost ×11
AI Agents That Matter (Kapoor et al., 2025)	93.2% for \$2.45 vs 88.0% for \$134.50 (run totals, stated)	simpler agent, better and cheaper

SET-UP COSTS LEFT OUT OF THE HEADLINES

Each needs n tasks to pay off

- JIT-Planner: 25–90 min of tool building plus 25–45 min of traces per app (Winston et al., 2026)
- EchoPath: a first pass of ≈ 572k tokens and ≈ 4.5 min per task (Zhao et al., 2026)
- AutoDroid-V2: \$82.42 of GPT-4o calls per app (calc.) (Wen et al., 2025)

References · Part 2 (1 of 6)

author-year tags on the Part 2 pages refer to these entries · a, b, c separate works whose first authors share a surname and year within Part 2 · published version where...

1. Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., & Zhang, Y. (2024). InferCept: Efficient intercept support for augmented large language model inference. *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, PMLR 235, 81–95. arXiv:2402.01869. University of California, San Diego.

2. Abhyankar, R., Qi, Q., & Zhang, Y. (2026). OSWorld-Human: Benchmarking the efficiency of computer-use agents. *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys 2026)*. arXiv:2506.16042. University of California, San Diego; GenseeAI.

3. Anthropic. (2026a, May 13). *Best practices for computer and browser use with Claude*. <https://claude.com/blog/best-practices-for-computer-and-browser-use-with-claude>

4. Anthropic. (2026b). *Claude API pricing; Prompt caching; Fast mode; Vision* (developer documentation, read 28 September 2026). <https://platform.claude.com/docs/en/about-claude/pricing>; <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>; <https://platform.claude.com/docs/en/build-with-claude/fast-mode>; <https://platform.claude.com/docs/en/build-with-claude/vision>

5. Automation Anywhere. (2026). *Automator AI* [Product page, undated, read 28 September 2026]; *Generative Recorder and resilient automation | April 2024* [Community Product Club recap, 13 May 2024]. <https://www.automationanywhere.com/products/automator-ai>; <https://community.automationanywhere.com/generative-recorder-85080/generative-recorder-and-resilient-automation-april-2024-88211> — vendor marketing (the product page’s 60% claims); the recap’s “nearly 50%” is vendor-stated, undefined.

6. Awadallah, A., Lara, Y., Magazine, R., Mozannar, H., Nambi, A., Pandya, Y., Rajeswaran, A., Rosset, C., Taymanov, A., Vineet, V., Whitehead, S., & Zhao, A. (2025). Fara-7B: An efficient agentic model for computer use. arXiv:2511.19663v1. Microsoft (Microsoft Research AI Frontiers; no affiliation line printed).

7. Bai, H., Lv, W., Zheng, H., Lu, Y., & Shu, J. (2026). SPORK: Self-speculative forking to accelerate agentic LLM inference. arXiv:2607.03333v1. Tsinghua University; Meituan.

8. Browserbase. (2026a). *How caching works in Stagehand (and where it breaks)* [Blog post, by S. Arif, 24 February 2026]; *Caching actions*, Stagehand v3 and v4 documentation (undated) (read 28 September 2026). <https://www.browserbase.com/blog/stagehand-caching>; <https://docs.stagehand.dev/v3/best-practices/caching>; <https://docs.stagehand.dev/v4/best-practices/caching> — the docs describe the mechanism; the blog’s speed-up figures are vendor marketing.

9. Browserbase. (2026b). *Pricing* (read 28 September 2026). <https://www.browserbase.com/pricing> — vendor pricing.

10. Cai, T., Wang, X., Ma, T., Chen, X., & Zhou, D. (2024). Large language models as tool makers (LATM). *The Twelfth International Conference on Learning Representations (ICLR 2024)*. arXiv:2305.17126. Google DeepMind; Princeton University; Stanford University.

11. Cerebras. (2026, August 13). *Accelerating GPT-5.6 Sol Ultrafast* [Blog post, by J. Er]. <https://www.cerebras.ai/blog/accelerating-gpt-5-6-sol-ultrafast-with-openai> — vendor and hardware-partner marketing.

12. Chen, B., Shu, C., Shareghi, E., Collier, N., Narasimhan, K., & Yao, S. (2023). FireAct: Toward language agent fine-tuning. arXiv:2310.05915v1. System2 Research; University of Cambridge; Monash University; Princeton University.

13. Chen, G., Zhou, X., Shao, R., Lyu, Y., Zhou, K., Wang, S., Li, W., Li, Y., Qi, Z., & Nie, L. (2025). Less is more: Empowering GUI agent with context-aware simplification (SimpAgent). *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2025)*, 5901–5911. <https://doi.org/10.1109/ICCV51701.2025.00558>. arXiv:2507.03730. Harbin Institute of Technology (Shenzhen); Huawei Noah’s Ark Lab.

14. Chen, L., Zaharia, M., & Zou, J. (2024). FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. arXiv:2305.05176. Stanford University.

15. Choi, D., Park, K., Song, W., Dingliwal, S., Jayanthi, S. M., Shin, J., & Galstyan, A. (2026). IdleSpec: Exploiting idle time via speculative planning for LLM agents. arXiv:2605.22154v1. KAIST; Amazon AGI; Together AI.

16. Chung, J., Shin, B., Kim, J., & Rhu, M. (2026). Agent-X: Full pipeline acceleration of on-device AI agents. *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services (MobiSys 2026)*. <https://doi.org/10.1145/3745756.3809195>. arXiv:2605.10380. KAIST.

17. Cuadron, A., Li, D., Ma, W., Wang, X., Wang, Y., Zhuang, S., ... Gonzalez, J. E. (2025). The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks. arXiv:2502.08235v1. University of California, Berkeley; ETH Zurich; University of Illinois Urbana-Champaign; Carnegie Mellon University.

18. DeepSeek. (2026). *Models & pricing* (API documentation, read 28 September 2026); *DeepSeek-V4.1-Flash release* (10 September 2026). https://api-docs.deepseek.com/quick_start/pricing; <https://api-docs.deepseek.com/news/news260910>

19. Dihan, M. L., Hashem, T., Ali, M. E., & Parvez, M. R. (2025). WebOperator: Action-aware tree search for autonomous agents in web environment. arXiv:2512.12692v1. Bangladesh University of Engineering and Technology; Monash University; Qatar Computing Research Institute.

References · Part 2 (2 of 6)

continued

20. Dong, Y., He, J., Liu, S., Hou, Y., Du, D., Xu, Z., Yu, S., Yang, B., Xia, Y., & Chen, H. (2026). DeltaBox: Scaling stateful AI agents with millisecond-level sandbox checkpoint/rollback. arXiv:2605.22781v2. Shanghai Jiao Tong University (IPADS); Huawei Technologies.
21. Enomoto, M., Obara, R., Zhang, H., & Oyamada, M. (2026). Revisiting observation reduction for web agents: Comprehensive evaluation with a lightweight framework. arXiv:2605.29397v1. NEC Corporation.
22. Erol, M. H., El, B., Suzgun, M., Yuksekgonul, M., & Zou, J. (2026). Cost-of-Pass: An economic framework for evaluating language models. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2504.13359v2. Stanford University.
23. Farzaneh, A., & Simeone, O. (2026). Think short, defer smart, act, and repeat: Calibrated reasoning and uncertainty-aware deferral for edge LLM agents. arXiv:2607.26865v3. Northeastern University London.
24. Feng, G., Mao, H., Dutta, P., & Gonzalez, J. E. (2026). Concurrency without model changes: Future-based asynchronous function calling for LLMs (AsyncFC). arXiv:2605.15077v1. University of California, Berkeley.
25. Gim, I., Lee, S., & Zhong, L. (2024). Asynchronous LLM function calling (AsyncLM). arXiv:2412.07017v1. Yale University.
26. Google. (2026). *Gemini Developer API pricing; Priority inference* (Gemini API documentation, updated 24 and 23 September 2026; read 28 September 2026). <https://ai.google.dev/gemini-api/docs/pricing>; <https://ai.google.dev/gemini-api/docs/priority-inference>
27. Guan, Y., Lan, Q., Sun, F., Ding, D., Acharya, D., Wang, C., Wang, W. Y., & Hua, W. (2026). Dynamic speculative agent planning. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2509.01920. Johns Hopkins University; University of Alberta; University of British Columbia; Avey Research Center; Google DeepMind; University of California, Santa Barbara.
28. Guo, Y., Xiao, Y., Zhang, J. M., Harman, M., Lou, Y., Liu, Y., & Chen, Z. (2026). EET: Experience-driven early termination for cost-efficient software engineering agents. *Findings of the Association for Computational Linguistics: ACL 2026*, 33008–33024. <https://doi.org/10.18653/v1/2026.findings-acl.1652>. arXiv:2601.05777. Nanyang Technological University; King’s College London; University College London; University of Illinois Urbana-Champaign; Tsinghua University.
29. Hajimiri, S., Aminbeidokhti, M., Dolz, J., Ben Ayed, I., Laradji, I. H., Gella, S., & Gontier, N. (2026). Are online skill and memory modules always worth their tokens? A budget-constrained study of web agents. arXiv:2606.15017v2 (arXiv comment: “Accepted to EMNLP 2026”; not yet on an official EMNLP 2026 list). ServiceNow AI Research; ÉTS Montréal; University of British Columbia; McGill University.
30. Hua, W., Wan, M., Vadrevu, S., Nadel, R., Zhang, Y., & Wang, C. (2025). Interactive speculative planning: Enhance agent efficiency through co-design of system and user interface. *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. arXiv:2410.00079. Rutgers University; Microsoft; Google DeepMind.
31. Huang, K.-H., Qiu, H., Dai, Y., Xiong, C., & Wu, C.-S. (2026a). GUI-KV: Efficient GUI agents via KV cache with spatio-temporal awareness. *Transactions on Machine Learning Research*. arXiv:2510.00536. Salesforce AI Research; University of California, Los Angeles.
32. Huang, Y., Srivatsa, V., Asch, A., Patwa, H. T., & Zhang, Y. (2026b). TClone: Low-latency forking of live GUI environments for computer-use agents. arXiv:2605.17320v1. University of California, San Diego; GenseeAI.
33. Huang, Z., Wang, X., Wang, A., Jurayj, W., Jiménez Gutiérrez, B., Khashabi, D., & Andrews, N. (2026c). Better, faster, stronger: Programmatic skill learning best reduces agent cost (SpeedRunner). arXiv:2608.11338v1. Johns Hopkins University.
34. Hyperbrowser. (2026). *Action caching* (HyperAgent documentation, undated, read 28 September 2026); *HyperAgent* [Source code], README.md on the main branch. <https://www.hyperbrowser.ai/docs/hyperagent/action-cache>; <https://github.com/hyperbrowserai/HyperAgent> — vendor documentation; “near-instant replay” is vendor marketing.
35. Jia, J., & Li, Q. (2026). AutoTool: Efficient tool selection for large language model agents. *Proceedings of the AAAI Conference on Artificial Intelligence, 40(37)*, 31265–31273. <https://doi.org/10.1609/aaai.v40i37.40389>. arXiv:2511.14650. Huazhong University of Science and Technology.
36. Jiang, W., Zhuang, Y., Song, C., Yang, X., Zhou, J. T., & Zhang, C. (2025). AppAgentX: Evolving GUI agents as proficient smartphone users. arXiv:2503.02268v3. Westlake University; Henan University; Southeast University; A*STAR (IHPC; CFAR).
37. Kang, H., Li, Z., Yang, X., Xu, W., Chen, Y., Wang, J., Chen, B., Krishna, T., Xu, C., & Arora, S. (2026). ThunderAgent: A fast, simple, and program-aware agentic inference system. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, poster. arXiv:2602.13692. Georgia Institute of Technology; University of Illinois Urbana-Champaign; Carnegie Mellon University; Together AI; independent researcher.
38. Kapoor, S., Stroebel, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2025). AI agents that matter. *Transactions on Machine Learning Research*. arXiv:2407.01502. Princeton University.

References · Part 2 (3 of 6)

continued

39. Kerboua, I., Omid Shayegan, S., Lù, X. H., Boisvert, L., Thakkar, M., Caccia, M., Espinas, J., Aussem, A., Eglin, V., & Lacoste, A. (2026). FocusAgent: Simple yet effective ways of trimming the large context of web agents. *Transactions on Machine Learning Research*. arXiv:2510.03204. Esker; INSA Lyon; ServiceNow Research; Mila; McGill University; Polytechnique Montréal; Université Lyon 1 (LIRIS).
40. Kerboua, I., Omid Shayegan, S., Thakkar, M., Lù, X. H., Caccia, M., Eglin, V., Aussem, A., Espinas, J., & Lacoste, A. (2025). LineRetriever: Planning-aware observation reduction for web agents. arXiv:2507.00210v1. INSA Lyon; Esker; ServiceNow Research; Mila; McGill University; Université Lyon 1 (LIRIS).
41. Kim, H., Wu, Y., & Tambe, T. (2026). AgenticCache: Cache-driven asynchronous planning for embodied AI agents. *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys 2026)*. arXiv:2604.24039. Seoul National University; Stanford University.
42. Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., & Gholami, A. (2024). An LLM compiler for parallel function calling. *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, PMLR 235. arXiv:2312.04511. University of California, Berkeley; International Computer Science Institute; Lawrence Berkeley National Laboratory.
43. Lai, H., Liu, X., Zhao, Y., Xu, H., Zhang, H., Jing, B., Ren, Y., Yao, S., Dong, Y., & Tang, J. (2026). ComputerRL: Scaling end-to-end online reinforcement learning for computer use agents. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2508.14040. Tsinghua University; Z.AI; University of Chinese Academy of Sciences.
44. Li, G., Wu, R., Tan, H., & Chen, G. (2024). A plan reuse mechanism for LLM-driven agent. *Journal of Computer Research and Development*, 61(11), 2706–2720. <https://doi.org/10.7544/issn1000-1239.202440380>. English version: Li, G., Wu, R., & Tan, H. (2025), arXiv:2512.21309v2. University of Science and Technology of China.
45. Li, H., He, R., Mang, Q., Zhang, Q., Mao, H., Chen, X., Zhou, H., Zhang, H., Cheung, A., Gonzalez, J., & Stoica, I. (2026a). Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv:2511.02230v7. University of California, Berkeley; Stanford University; Tsinghua University.
46. Li, K., Meng, Z., Lin, H., Luo, Z., Tian, Y., Ma, J., Huang, Z., & Chua, T.-S. (2025). ScreenSpot-Pro: GUI grounding for professional high-resolution computer use. *Proceedings of the 33rd ACM International Conference on Multimedia (MM 2025)*, 8778–8786. <https://doi.org/10.1145/3746027.3755688>. arXiv:2504.07981. National University of Singapore; East China Normal University; Hong Kong Baptist University.
47. Li, T., Hu, J., Wang, Y., Liu, J., & Liu, X. (2026b). WebRouter: Query-specific router via variational information bottleneck for cost-sensitive web agent. *ICASSP 2026 – IEEE International Conference on Acoustics, Speech and Signal Processing*, 4086–4090. <https://doi.org/10.1109/ICASSP55912.2026.11464950>. arXiv:2510.11221. Nanjing University of Aeronautics and Astronautics; Hong Kong Baptist University; Beihang University; Pengcheng Laboratory.
48. Li, Y., Ye, Q., Choubey, P. K., Zhang, J., & Wu, C.-S. (2026c). Speculate with memory: Lossless acceleration for LLM agents. arXiv:2607.12236v1. Salesforce Research.
49. Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., & Qiu, L. (2024). Parrot: Efficient serving of LLM-based applications with semantic variable. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Shanghai Jiao Tong University; Microsoft Research.
50. Lin, X., Liew, J. H., Savarese, S., & Li, J. (2026). W&D: Scaling parallel tool calling for efficient deep research agents. arXiv:2602.07359v1. Salesforce AI Research.
51. Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., & Zharov, Y. (2025). The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. Paper presented at the 4th Deep Learning for Code Workshop (DL4C): Deep Learning for Code in the Agentic Era, NeurIPS 2025. arXiv:2508.21433v3. JetBrains Research; Technical University of Munich.
52. Liu, T., Wang, Z., Miao, J., Hsu, I.-H., Yan, J., Chen, J., ... Lee, C.-Y. (2026a). Budget-aware tool use enables effective agent scaling (Budget Tracker, BATS). *Third Conference on Language Modeling (COLM 2026)*. arXiv:2511.17006. University of California, Santa Barbara; Google Cloud AI Research; Google DeepMind; New York University.
53. Liu, Y., Huang, Y., Yao, J., Feng, S., Gu, Z., Du, K., Li, H., Cheng, Y., Jiang, J., Lu, S., Musuvathi, M., & Choukse, E. (2026b). DroidSpeak: KV cache sharing across fine-tuned model variants. *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, 319–338. arXiv:2411.02820 (under another title). University of Chicago; Microsoft.
54. Liu, Z., Kundu, S., & Beerel, P. A. (2026c). Speculative macro commit for faster tool-using agents. arXiv:2609.03236v1 (arXiv comment: “Accepted in MLSP2026”). University of Southern California; Intel Labs.
55. Lu, J., Zhang, Z., Yang, F., Zhang, J., Wang, L., Du, C., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025a). AXIS: Efficient human-agent-computer interaction with API-first LLM-based agents. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7711–7743. <https://doi.org/10.18653/v1/2025.acl-long.381>. Peking University; Nanjing University; Microsoft.
56. Lu, Q., Ding, L., Cao, S., Liu, X., Zhang, K., Zhang, J., & Tao, D. (2025b). Runaway is ashamed, but helpful: On the early-exit behavior of large language model-based agents in embodied environments. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 24014–24027. <https://doi.org/10.18653/v1/2025.findings-emnlp.1304>. arXiv:2505.17616. Southeast University; The University of Sydney; Harbin Institute of Technology (Shenzhen); Nanyang Technological University.
57. Lumer, E., Nizar, F., Jangiti, A., Frank, K., Gulati, A., Phadate, M., & Subbiah, V. K. (2026). Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks. arXiv:2601.06007v2. PricewaterhouseCoopers (PwC U.S.).

References · Part 2 (4 of 6)

continued

58. Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., & Stoica, I. (2026). Agentix: An efficient serving engine for LLM agents as general programs. *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. University of California, Berkeley; Google DeepMind; Shanghai Jiao Tong University.
59. Microsoft. (2026a). *Enable priority processing for Microsoft Foundry models* (Microsoft Learn, 22 September 2026); *Azure OpenAI pricing* (prices taken from the page's data attributes, US East) (read 28 September 2026). <https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/priority-processing>; <https://azure.microsoft.com/en-us/pricing/details/azure-openai/>
60. Microsoft. (2026b). *Self-healing (preview)* (Power Automate documentation, Microsoft Learn, 30 June 2026, updated 11 July 2026); *FAQ for Repair with Copilot at runtime in Power Automate desktop* (16 January 2026) (read 28 September 2026). <https://learn.microsoft.com/en-us/power-automate/desktop-flows/self-healing>; <https://learn.microsoft.com/en-us/power-automate/faqs-repair-copilot> — vendor documentation.
61. Mohammadi, B., Klein, L., Arora, A., & Bindschaedler, L. (2026a). Ghost tool calls: Issue-time privacy for speculative agent tools. arXiv:2606.02483v1. Max Planck Institute for Software Systems; EPFL; Aarhus University.
62. Mohammadi, B., Potamitis, N., Klein, L., Arora, A., & Bindschaedler, L. (2026b). Atomix: Timely, transactional tool use for reliable agentic workflows. arXiv:2602.14849v2. Max Planck Institute for Software Systems; Aarhus University; EPFL.
63. Nichols, D., Singhanian, P., Jekel, C., Bhatele, A., & Menon, H. (2025). Optimizing agentic language model inference via speculative tool calls. arXiv:2512.15834v1. Lawrence Livermore National Laboratory; University of Maryland.
64. Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2025). RouteLLM: Learning to route LLMs from preference data. *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. arXiv:2406.18665. University of California, Berkeley; Anyscale; Canva.
65. OpenAI. (2026). *API pricing; API changelog* (Priority processing, 27 June 2025; Fast mode, 30 July and 5 August 2026; Ultrafast, 13 August 2026); *Fast mode, Flex processing, Batch API and Prompt caching* guides (developer documentation, read 28 September 2026); *Prompt Caching 201* (OpenAI Cookbook, 18 February 2026). <https://developers.openai.com/api/docs/pricing>; <https://developers.openai.com/api/docs/changelog>; <https://developers.openai.com/api/docs/guides/fast-mode>; <https://developers.openai.com/api/docs/guides/flex-processing>; <https://developers.openai.com/api/docs/guides/batch>; <https://developers.openai.com/api/docs/guides/prompt-caching>; https://developers.openai.com/cookbook/examples/prompt_caching_201
66. Pan, Z., Patel, A., Shen, Y., Hu, Z., Guan, Y., Li, W.-L., Qin, L., Wang, Y., & Ding, Y. (2025). KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 139912–139931. <https://doi.org/10.52202/085713-4208>. arXiv:2507.07400. University of California, San Diego; Amazon Web Services.
67. Prabhu, V., Dai, Y., Fernandez, M., Ramakrishnan, K., Gu, J., Luo, Y., Savarese, S., Xiong, C., Li, J., Chen, Z., & Xu, R. (2026). WALT: Web agents that learn tools. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2510.01524. Salesforce AI Research.
68. Qin, Z., Yue, S., Hua, X., Fu, Y., & Ren, J. (2026). Executable agentic memory for GUI agent. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*. arXiv:2605.12294. Tsinghua University; Sun Yat-sen University.
69. Shen, J., Bai, H., Zhang, L., Zhou, Y., Setlur, A., Tong, S., Caples, D., Jiang, N., Zhang, T., Talwalkar, A., & Kumar, A. (2025). Thinking vs. doing: Improving agent reasoning by scaling test-time interaction. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 187840–187881. <https://doi.org/10.52202/085713-5647>. arXiv:2506.07976 (under another title). Carnegie Mellon University; Scribe; University of Illinois Urbana-Champaign; University of Toronto; University of California, Berkeley; The AGI Company; New York University.
70. Shi, C., Chen, Y., Chen, Y., Miao, X., & Cui, B. (2026a). Multi-segment attention: Enabling efficient KV-cache management for faster large language model serving (AsynCache). arXiv:2606.02964v1. Peking University.
71. Shi, H., Wang, W., Fang, W., Liang, Y., Jin, T., Zhao, P., Liu, G., Chen, S., & Wang, Y. (2026b). CUA-Universe: A scalable and dynamic environment for hybrid GUI+CLI agents. arXiv:2609.05374v1. Shanghai Jiao Tong University; Zhejiang University.
72. Skyvern. (2026). *Code caching; Cost control* (developer documentation, undated, read 28 September 2026). <https://www.skyvern.com/docs/developers/features/code-caching>; <https://skyvern.mintlify.app/developers/optimization/cost-control> — vendor documentation.
73. Song, L., Dai, Y., Prabhu, V., Zhang, J., Shi, T., Li, L., Li, J., Savarese, S., Chen, Z., Zhao, J., Xu, R., & Xiong, C. (2026). CoAct-1: Computer-using multi-agent system with coding actions. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. arXiv:2508.03923. University of Southern California; Salesforce; University of Washington.
74. Song, Y., Xu, F. F., Zhou, S., & Neubig, G. (2025). Beyond browsing: API-based web agents. *Findings of the Association for Computational Linguistics: ACL 2025*, 11066–11085. <https://doi.org/10.18653/v1/2025.findings-acl.577>. arXiv:2410.16464. Carnegie Mellon University.
75. Srivatsa, V., He, Z., Abhyankar, R., Li, D., & Zhang, Y. (2025). Preble: Efficient distributed prompt scheduling for LLM serving. *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. arXiv:2407.00023. University of California, San Diego.
76. Tang, F., Shen, Y., Zhang, H., Chen, S., Hou, G., Zhang, W., Zhang, W., Song, K., Lu, W., & Zhuang, Y. (2025). Think twice, click once: Enhancing GUI grounding via fast and slow systems (FOCUS). arXiv:2503.06470v1. Zhejiang University; Microsoft Research Asia.

References · Part 2 (5 of 6)

continued

77. UiPath. (2026). *Healing Agent user guide: What is Healing Agent?; Recovery strategies; Licensing; Frequently asked questions* (Automation Cloud documentation, undated, read 28 September 2026). <https://docs.uipath.com/agents/automation-cloud/latest/user-guide-ha/what-is-healing-agent>; <https://docs.uipath.com/agents/automation-cloud/latest/user-guide-ha/deterministic-recovery-strategies>; <https://docs.uipath.com/agents/automation-cloud/latest/user-guide-ha/licensing>; <https://docs.uipath.com/agents/automation-cloud/latest/user-guide-ha/frequently-asked-questions> — vendor documentation.

78. Wadlom, N., Shen, J., & Lu, Y. (2026). Efficient LLM serving for agentic workflows: A data systems perspective (Helium). *Proceedings of the ACM on Management of Data*, 4 (SIGMOD 2026). <https://doi.org/10.1145/3802046>. arXiv:2603.16104v1 (extended version). National University of Singapore.

79. Wang, N., Hu, X., Liu, P., Zhu, H., Hou, Y., Huang, H., ... Zhou, W. (2025a). Efficient agents: Building effective agents while reducing cost. arXiv:2508.02694v1. OPPO (OPPO AI Agent Team).

80. Wang, Z. Z., Gandhi, A., Neubig, G., & Fried, D. (2025b). Inducing programmatic skills for agentic tasks (ASI). *Second Conference on Language Modeling (COLM 2025)*. arXiv:2504.06821. Carnegie Mellon University.

81. Wang, Z. Z., Mao, J., Fried, D., & Neubig, G. (2025c). Agent workflow memory. *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, PMLR 267, 63897–63911. arXiv:2409.07429. Carnegie Mellon University; Massachusetts Institute of Technology.

82. Wei, J., Ni, K., Zhao, Y., Gan, G., & Cohan, A. (2026). Step-level optimization for efficient computer-use agents (StepWise). arXiv:2604.27151v1. Yale University; University of North Carolina at Chapel Hill.

83. Wen, H., Tian, S., Pavlov, B., Du, W., Li, Y., Chang, G., Zhao, S., Liu, J., Liu, Y., Zhang, Y.-Q., & Li, Y. (2025). AutoDroid-V2: Boosting SLM-based GUI agents via code generation. *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services (MobiSys 2025)*, 223–235. <https://doi.org/10.1145/3711875.3729134>. arXiv:2412.18116. Tsinghua University (AIR); Shanghai AI Laboratory; Beijing Academy of Artificial Intelligence.

84. Winston, C., Wang, R. Y., Mirhoseini, A., & Kozyrakis, C. (2026). Agent JIT compilation for latency-optimizing web agent planning and scheduling. *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, PMLR 306. arXiv:2605.21470. Stanford University.

85. Wong, M., Hsieh, K., Nath, S., & Netravali, R. (2026). Skim: Speculative execution for fast and efficient web agents. arXiv:2605.16565v2. Princeton University; Microsoft Research.

86. Wu, G., Li, D., Jiang, K., Niu, J., Wang, C., & Zhang, Y. (2026). Safe to resume? Breaking execution continuity of agent execution via rollback. arXiv:2608.29381v1. Southern University of Science and Technology; City University of Hong Kong.

87. xAI. (2026). *Grok 4.7 model page*, SpaceXAI Docs (read 28 September 2026). <https://docs.x.ai/developers/models/grok-4.7>

88. Xia, H., Li, Y., Du, C., Song, M., & Li, W. (2026). ToolSpec: Accelerating tool calling via schema-aware and retrieval-augmented speculative decoding. arXiv:2604.13519v2. The Hong Kong Polytechnic University; Peking University.

89. Xiao, Y.-A., Gao, P., Peng, C., & Xiong, Y. (2026). Reducing cost of LLM agents with trajectory reduction (AgentDiet). *Proceedings of the ACM on Software Engineering*, 3(FSE), Article FSE056. <https://doi.org/10.1145/3797084>. arXiv:2509.23586. Peking University; ByteDance.

90. XLANG Lab. (2026). OSWorld 2.0: Benchmarking computer use agents on long-horizon real-world tasks. arXiv:2606.29537v2. The University of Hong Kong (authored as “XLANG Lab and Collaborators”; the full author list is in the paper’s Appendix A).

91. Xu, B., Xue, Z., Chen, D., Fu, C., Wu, C., Huang, C., ... Zhang, N. (2026a). TokenPilot: Cache-efficient context management for LLM agents. arXiv:2606.17016v2 (arXiv comment: “EMNLP 2026 Findings”; not yet on an official EMNLP 2026 list). Zhejiang University; University of Electronic Science and Technology of China; Xidian University; HomologyAI.

92. Xu, H., Huang, X., Liu, Y., & Deng, Z. (2026b). TPS-Bench: Evaluating AI agents’ tool planning & scheduling abilities in compounding tasks. *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 34949–34961. <https://doi.org/10.18653/v1/2026.acl-long.1614>. arXiv:2511.01527. Shanghai Jiao Tong University.

93. Xu, Y., Wang, Z., Wang, J., Lu, D., Xie, T., Saha, A., Sahoo, D., Yu, T., & Xiong, C. (2025). Aguviz: Unified pure vision agents for autonomous GUI interaction. *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, PMLR 267. arXiv:2412.04454. The University of Hong Kong; Salesforce Research.

94. Yang, J., Hou, B., Wei, W., Bao, Y., & Chang, S. (2026a). Ares: Adaptive reasoning effort selection for efficient LLM agents. arXiv:2603.07915v1. University of California, Santa Barbara; Accenture.

95. Yang, K., Liu, Y., Chaudhary, S., Fakoor, R., Chaudhari, P., Karypis, G., & Rangwala, H. (2025). AgentOccam: A simple yet strong baseline for LLM-based web agents. *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. arXiv:2410.13825. University of Illinois Urbana-Champaign; Amazon.

References · Part 2 (6 of 6)

continued

96. Yang, Y., Jin, C., Zhao, J., Wu, J., Zhou, Y., Wang, Z., Wang, Z., Zhou, M., & Metaxas, D. N. (2026b). Act more, decide less: Skill-guided adaptive action chunking for long-horizon LLM agents (SPACE). arXiv:2609.02042v1 (arXiv comment: “EMNLP 2026 Camera Ready”; not yet on an official EMNLP 2026 list). Rutgers University; University of Toronto; The Hong Kong Polytechnic University; Amazon; Microsoft.

97. Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., & Jiang, J. (2025). CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys 2025)*, 94–109. <https://doi.org/10.1145/3689031.3696098>. arXiv:2405.16444. University of Chicago; The Chinese University of Hong Kong, Shenzhen; Stanford University; Microsoft Research.

98. Ye, H., Gao, Z., Ma, M., Wang, Q., Fu, Y., Chung, M.-Y., Lin, Y., Liu, Z., Zhang, J., Zhuo, D., & Chen, Y. (2025). KVCOMM: Online cross-context KV-cache communication for efficient LLM-based multi-agent systems. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. <https://doi.org/10.52202/085713-0605>. arXiv:2510.12872. Duke University; Massachusetts Institute of Technology; NVIDIA.

99. Ye, N., Ahuja, A., Liargkovas, G., Lu, Y., Kaffes, K., & Peng, T. (2026). Speculative actions: A lossless framework for faster AI agents. *The Fourteenth International Conference on Learning Representations (ICLR 2026)*, oral. arXiv:2510.04371. Columbia University.

100. Yu, Z., Li, Q., Gao, Z., Xiang, L., Shi, C., Liu, Y., Wu, H., Wei, Y., Fei, Y., Fu, Y., & He, Z. (2026). Beyond sequential interaction: Benchmarking parallel execution and coordination for GUI agents (ParaGUIBench). arXiv:2607.22689v1. Beijing University of Posts and Telecommunications; BIGAI; China University of Geosciences (Beijing); Beijing Institute of Technology.

101. Zhai, Y., Shen, D., Luo, J., & Yang, B. (2026). ToolCaching: Towards efficient caching for LLM tool-calling. arXiv:2601.15335v1. Southeast University.

102. Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., ... Zhang, D. (2026a). UF02: The desktop AgentOS. *Transactions on Machine Learning Research*. arXiv:2504.14603. Microsoft; ZJU-UIUC Institute; Nanjing University; Peking University.

103. Zhang, C., Xia, M., Zhang, X., Madrigal, D., Mallick, A., Kessler, S., Rühle, V., & Rajmohan, S. (2026b). Budget-aware agentic routing via boundary-guided training (BoPO). arXiv:2602.21227v1. University of Cambridge; Microsoft (M365 Research).

104. Zhang, J., Chen, K., Lu, Z., Zhou, E., Yu, Q., & Zhang, J. (2026c). Prune4Web: DOM tree pruning programming for web agent. *Proceedings of the AAAI Conference on Artificial Intelligence, 40*(41), 34710–34718. <https://doi.org/10.1609/aaai.v40i41.40772>. arXiv:2511.21398. Beihang University.

105. Zhang, L., Gao, L., & Xu, M. (2026d). Does chain-of-thought reasoning help mobile GUI agents? An empirical study. *Findings of the Association for Computational Linguistics: ACL 2026*, 7981–7996. <https://doi.org/10.18653/v1/2026.findings-acl.392>. Beijing University of Posts and Telecommunications; Tsinghua University.

106. Zhang, Q., Wornow, M., & Olukotun, K. (2025). Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. <https://doi.org/10.52202/085713-3451>. arXiv:2506.14852 (v2 adds a fourth author, G. Wan). Stanford University.

107. Zhao, Y., Shanmugham, A., Roy, S., & Xu, Y. (2026). EchoPath: Execution-level replayable memory for GUI agents. arXiv:2609.16635v1. Johns Hopkins University; Amazon AGI.

108. Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. <https://doi.org/10.52202/079017-2000>. arXiv:2312.07104. Stanford University; University of California, Berkeley; Shanghai Jiao Tong University; Texas A&M University; independent researcher.

109. Zhong, H., Faisal, F., França, L., Leesatapornwongsa, T., Szekeres, A., Rong, K., & Nath, S. (2026). ActionEngine: From reactive to programmatic GUI agents via state machine memory. arXiv:2602.20502v1. Georgia Institute of Technology; Microsoft Research.

110. Zhou, Y., Dai, S., Wang, S., Zhou, K., Jia, Q., & Xu, J. (2025). GUI-G1: Understanding R1-Zero-like training for visual grounding in GUI agents. *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. <https://doi.org/10.52202/085713-3201>. arXiv:2505.15810. Renmin University of China; Huawei Noah’s Ark Lab.

111. Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., & Kasikci, B. (2026). TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560v2. University of Washington; Wuhan University of Technology; Shanghai Jiao Tong University. Dashboard: <https://tracelab.cs.washington.edu> (read 28 September 2026).

B0 · Source formulas for overlap: how much concurrency can save

verbatim from each source, in its own symbols; the right-hand column maps them onto Part 1 · behind pages 9–10

SOURCE, LOCATION	THE FORMULA IN THE SOURCE’S OWN SYMBOLS	ITS SYMBOLS	WHAT IT BOUNDS, IN PART 1’S TERMS
AsyncFC, §4 Eq. 1 (= App. B Eq. 2) (Feng et al., 2026)	$R = \frac{T_{\text{LLM}} + T_{\text{tool}}}{\max(T_{\text{LLM}}, T_{\text{cp}})}$	T_{LLM} total decoding time; T_{tool} total function time; T_{cp} function time on the critical path of the dependency graph	the largest speed-up that T_{saving} can give; decoding sits in D_i , functions in E_i ; its R is not the success probability $R_m(p)$ (page 9 writes “speed-up”)
LLMCompiler, App. E.1 (Kim et al., 2024)	$T^R = \sum_{i=1}^N (T_P^R(P_i) + T_E(E_i))$ $T^C = \sum_{i=1}^N T_P^C(P_i) + \max_k T_E(E_k)$ $\gamma = T^R / T^C$	N planned tasks (not our steps); P_i planning and E_i execution of task i (not our E_i); R: ReAct, C: compiled	serial against parallel execution; $\gamma_{\text{max}} \approx N$ when execution dominates and all calls take equally long, ≈ 1 when planning dominates
Speculative tool calls, Eq. 2 and Lemma 1 (Nichols et al., 2025)	$S_{\text{spec}} = \frac{G + T}{\alpha \max\{G, g + T\} + (1 - \alpha)(G + T)}$ $S_{\text{max}} = \frac{G + T}{\max\{G, g + T\}} < 2$	G mean generation time of the main model; g of the speculator; T mean tool time; α acceptance rate	one model call overlapped with one tool call: under 2× per step, whatever the guesser
SPORK, EQ1 and App. A (Bai et al., 2026)	$\text{Ratio} = \frac{T_{\text{base}}}{T_{\text{base}}^* - \alpha t_{\text{overlap}} + T_{\text{oh}}}$ $S_{\text{max}} = \frac{1}{1 - f_{\text{tool}}}$	$T_{\text{base}} = T_{\text{dec}} + T_{\text{tool}}$; $T_{\text{base}}^* = T_{\text{base}} - T_{D3}$, less the decoding saved by its third design (D3: a rejected probe’s prefix reused as draft tokens); α accepted share of tool-call turns; t_{overlap} overlap per accepted turn; T_{oh} probe overhead; $f_{\text{tool}} = T_{\text{tool}} / T_{\text{base}}$	break-even: $\alpha t_{\text{overlap}} \geq T_{\text{oh}}$ (without its third design); the ceiling is Amdahl’s law on the tool share
ParaGUI, §III-C (Yu et al., 2026)	$L_{\text{parallel}} = \sum_{r=1}^R \max_k s_{r,k}$ $S = L_{\text{serial}} / L_{\text{parallel}}, \quad P = L_{\text{total}} / L_{\text{parallel}}$	$s_{r,k}$ GUI steps of worker k in round r; R rounds (not $R_m(p)$); L_{serial} steps of a single-agent serial run; L_{total} all worker steps	steps on the critical path against total work: P = 2.13 means 2.13 desktops busy on average — machine hours x_{env} for time

B0 · Source formulas for speculation, cache retention and a reader call

verbatim from each source, in its own symbols · behind pages 5, 6 and 10

SOURCE, LOCATION	THE FORMULA IN THE SOURCE’S OWN SYMBOLS	ITS SYMBOLS	WHAT IT BOUNDS, IN PART 1’S TERMS
Speculative Actions, Prop. 1 and Thm 3 (Ye et al., 2026)	$\frac{\mathbb{E}[T_s]}{\mathbb{E}[T_{\text{seq}}]} \rightarrow 1 - \frac{p(k)}{1 + p(k)} \cdot \frac{\alpha}{\alpha + \beta}$ $\frac{\mathbb{E}[M_{\text{spec}} - M_{\text{seq}}]}{\mathbb{E}[M_{\text{seq}}]} \rightarrow \tilde{k} - (\tilde{k} + \frac{\alpha}{\alpha + \beta}) \frac{p(k)}{1 + p(k)}$	$p(k) = 1 - (1 - p)^k$, k guesses of accuracy p; speculator and API latencies exponential with rates α, β ; T_s, T_{seq} time with and without speculation; $\rightarrow$ the limit as the number of steps grows; $\tilde{k}$ distinct guessed actions; M money	time saved and money added from the same guess accuracy; in the limit the time ratio stays above 1/2 (calc.), so at most 50% is saved — under its exponential-latency model
Speculate with memory, §2.1 and §4.3 (Li et al., 2026c)	saving per hit $= \min(\ell_{\text{env}}, \ell_{\text{LLM}} - \ell_{\text{spec}})$ (guess the action) or $\min(\ell_{\text{LLM}}, \ell_{\text{env}} - \ell_{\text{spec}})$ (guess the observation); extra cost $= k C_S + (1 - \text{acc}) C_{\text{act}}$	ℓ latencies of the environment, the model and the speculator; k speculators of cost C_S ; acc hit rate; C_{act} cost of the pre-launched actor call a miss wastes (the extra-cost form holds for observation and chained guesses)	a hit can hide at most the idle window of the other side; its App. A uses slightly different per-type forms
Continuum, Eqs. 1–2 (Li et al., 2026a)	$\tau^* = \arg \max_{\tau} \mathcal{P}(\tau, f) \text{Benefit}(r) - \text{Cost}(\tau, r)$	$\mathcal{P}(\tau, f)$ empirical probability that tool f returns within τ ; Benefit: reload and reordering cost avoided; Cost: memory held for τ	how long to keep a run’s cache through a tool wait: t^{queue} and re-prefill against GPU memory
InferCept, Eqs. 1–5 (Abhyankar et al., 2024)	$\text{Waste}_{\text{preserve}} = T_{\text{INT}} C_i^j M$; also discard, swap and chunked discard; $\text{Waste} = \min(\text{Waste}_{\text{preserve}}, \text{Waste}_{\text{chunkD}})$ (Eq. 5)	T_{INT} duration of the tool call; C context tokens; M memory per token; i request, j interception (not Part 1’s step and call)	the same trade inside the server: keep the cache during E_i , or pay t^{prefill} again
FocusAgent, App. H.1 (Kerboua et al., 2026)	$C_S o_i + C_L o_r \leq C_L o_i \Leftrightarrow \alpha \leq \frac{C_L - C_S}{C_L}$	C_S, C_L prices of the small reader and the large actor; $ o_i , o_r $ observation before and after reduction; $\alpha = o_r / o_i $	when a reader call pays in money; the paper notes it ignores API latency — the time can still grow (page 6)

B0 · Source formulas for money and set-up, and one closer source for Part 1

verbatim from each source · behind pages 12 and 14 · the last row is a finding for Part 1’s Appendix A0

SOURCE, LOCATION	THE FORMULA IN THE SOURCE’S OWN SYMBOLS	ITS SYMBOLS	WHAT IT BOUNDS, IN PART 1’S TERMS
TokenPilot, App. A.2, Eq. 11 (Xu et al., 2026a)	$\text{Cost} = C'_{\text{hit}} p_{\text{hit}} + C'_{\text{miss}} p_{\text{miss}} + H_{\text{out}} p_{\text{out}}$	$ C'_{\text{hit}} , C'_{\text{miss}} $ cached and uncached input tokens; H_{out} generated tokens; p prices	Part 1’s money sum over three classes; no cache-write term
SpeedRunner, App. A.3 (Huang et al., 2026c)	$\text{cost} = p_{\text{in}} N_{\text{in}}^{\text{uncached}} + p_{\text{cache}} N_{\text{in}}^{\text{cached}} + p_{\text{out}} N_{\text{out}}$	N summed over all calls of an episode, the skill-inducer calls amortised over its rollouts	the same three classes; set-up calls spread over the runs they serve
LATM, Table 2 (Cai et al., 2024)	$O(nc + C)$ against $O(nC)$	n samples; C one call to the large tool-making model; c one call to the small tool-using model	one expensive set-up amortised over n cheap runs — the form of page 14’s C_{setup}/n
AI Agents That Matter, §3 (Kapoor et al., 2025)	in words: total cost = a fixed cost (one-time optimisation) + a variable cost (per run, from input and output tokens)	—	page 14’s $v_n(m, p)$ adds this fixed cost to Cost-of-Pass’s per-success cost (adapted)
Speculative tool calls, Eq. 4 (Nichols et al., 2025)	$T_{\text{vanilla}} = 2Ko + \phi \sum_{i=1}^K X_i + \delta \sum_i (R_i + t_i) + \sum_i T_i, \quad X_{i+1} = X_i + t_i + t_{o,i}$	K turns; o per-call overhead; ϕ prefill and δ decode seconds per token; X_i prompt tokens of turn i; R_i, t_i reasoning and tool-call tokens; $t_{o,i}$ tool output; T_i tool time	a published form of Part 1’s time sum with its growing prompt, found in these checks: per-call overhead, prefill of a prompt that grows each turn, decoding, tool wait. Part 1 does not cite it yet (open question)

B1 · Compile or replay: the research systems, with full conditions

behind page 3 · term J_i → 0 on compiled or replayed steps · calc. = our arithmetic

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
JIT-Planner (Winston et al., 2026)	Candidate code plans over cached site tools with pre/postcondition checks, drafted by parallel planning calls; a separate scheduler (not in the headline) can hedge by running up to 4 copies of the chosen plan	Wall-clock from submission to completion, planning included: 150.1 → 15.4 s (9.7×, its Table 1), success 61 → 90%. 37 tasks in 5 apps (18 written by the authors), 3 runs, GPT-4.1; Browser-Use v0.7.10 on the same model. Three-model average 10.4×	Tool synthesis 25–90 min + traces 25–45 min per app; no tokens or \$ reported
ActionEngine (Zhong et al., 2026)	Offline state-machine graph of the site; one sketch-program call compiled by graph search	106 WebArena Reddit tasks: success 66 → 95%, end-to-end latency 237 → 118 s, model calls 10.2 → 1.8, input tokens 62.3k → 8.1k, \$0.71 → \$0.06 per task (tokens × list price). AgentOccam on GPT-4-Turbo vs ActionEngine on Claude Sonnet 4.5	About 2.5× of the 11.8× \$ gap is the cheaper model's price (calc.); crawl cost not reported
AutoDroid-V2 (Wen et al., 2025)	A local model writes a whole-task script from an offline app document	Model inference time 669.2 → 46.3 s per task (prompt to last token, screen actions excluded); uncached input 3,021 → 68 tokens; success 43.9 → 54.4%. 158 DroidTask tasks, fine-tuned Llama-3.1-8B on a phone, both sides	Offline GPT-4o \$4.11 + \$7.83 + \$70.48 = \$82.42 per app (calc.), ~2.5 GPU-hours of fine-tuning
EchoPath (Zhao et al., 2026)	Validated trajectories kept as executable memories; a gate checks the start state	Second pass, medians: tokens 586,386 → 20,370 (−96.5%), time 315.7 → 127.5 s (−59.6%), success 91.8 → 91.2% (calc. for the %). OSWorld-Verified, 159 replayable tasks, Codex GPT-5.5, vs Synapse; the second pass changes the resolution	First pass ≈ 572k tokens, ≈ 4.5 min per task; the gate accepted 22% of incompatible starts (calc.)
AppAgentX (Jiang et al., 2025)	Frequent action sequences evolved into shortcut actions	AndroidWorld, 116 tasks, GPT-4o: task time 147.17 → 59.74 s, tokens 18.9k → 6.2k, success 41.7 → 62.5%; time compared only on tasks every method completed	Evolution runs before the test
AutoTool (Jia & Li, 2026)	Repeated tool calls issued from a usage graph without a model call, capped at 30% of operations	Model calls 24.1 → 20.4 (ALFWorld), 23.3 → 17.8 (ScienceWorld); progress rate 0.394 → 0.531 and 0.716 → 0.708 — Llama4-Scout-17B	No wall-clock; calls used as the proxy
EAM (Qin et al., 2026)	Executable memory of sub-tasks for mobile agents	Per step 9.3 → 2.8 s and 50.8K → 8.3K tokens vs GPT-4o (benchmark not named); AndroidWorld success 34.5 → 52.6%, 3 runs	Per step, not per task
AgentReuse (Li et al., 2024)	Reuses stored plans for semantically similar requests	−93.12% latency is computed from an assumed 31.8 s per plan, not measured end to end	Modelled estimate; not used on the main page

B1 · Compile or replay in products: replay first, a model only to repair

behind page 3 · vendor documentation and pages, read 28 September 2026 · vendor figures are labelled as such; marketing is not evidence

PRODUCT	WHAT REPLAYS, AND WHEN A MODEL IS CALLED	PUBLISHED NUMBERS	WHAT IS MISSING
UiPath Healing Agent (UiPath, 2026)	Recorded automation; a model recommends or applies a fix when a step fails	Price only: one heal = 3 Platform Units; failed heals free; healing is not kept for the next run	Heal success rate, heal latency
Power Automate self-healing (Microsoft, 2026b)	Desktop flows repaired with Copilot	None; free on premium accounts	Any measured rate
Automation Anywhere Generative Recorder (Automation Anywhere, 2026)	Recorded bot with a generative fallback	“Nearly 50%” of failures fixed in customer previews: a vendor statement with no sample, period or definition (the product page’s “over 60%” is marketing, not cited)	A defined, sampled rate
Stagehand cache (Browserbase, 2026a)	An action’s model output is cached; the second run of the same action makes no call	None usable: the blog’s “as high as ~80%” (one action run twice) is vendor marketing, not cited	Whole-workflow numbers, sample, model
Skyvern code cache (Skyvern, 2026)	Generated code re-run without a model	None (“faster, cheaper”); plans imply about \$0.024 per action (calc.)	Any measured rate
HyperAgent action cache (Hyperbrowser, 2026)	Replay by XPath; the model only when the page has changed	None; the docs concede that a fallback adds latency and cost	Replay and fallback rates

Across all six: the model call is replaced by replay (**J_i** → **0**) and the environment actions stay (**E_i unchanged**); no vendor publishes a replay or repair success rate with a defined sample.

B2 · Fewer steps: more rows, with full conditions

behind page 4 · term N ↓ · steps are counted as each paper counts them · calc. = our arithmetic

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
UF02, GUI + API (Zhang et al., 2026a)	Native API calls replace GUI sequences in Office apps	Model-involved steps 16.0 → 6.6 with o1, but 13.8 → 12.9 with GPT-4o, on tasks both settings completed; 12 hand-built APIs	No wall-clock
CUA-Verse (Shi et al., 2026b)	A command-line tool added to a fine-tuned 9B agent	OSWorld subset of 244 tasks, same model: success 23.4 → 40.2%, mean steps 39.6 → 28.6, tokens 325.7K → 286.5K	Training on 4,923 episodes
SPACE (Yang et al., 2026b)	RL-trained variable-length action chunks	ALFWorld unseen split, Qwen3-4B: model rounds 20.9 → 4.4, success 81.3 → 96.9% vs the best baseline (multi-action GRPO)	Text environments only; no time or tokens
SpeedRunner (Huang et al., 2026c)	A coding agent refactors the skill library for cost	Crafter: model calls −94% over training; Gemini-3-Flash, 3 seeds	Text game; with Gemini-3-Flash success fell significantly below the OPO baseline on every benchmark
AWM (Wang et al., 2025c)	Induced workflows injected into the prompt	WebArena 812 tasks: success 15.0 → 35.5%, steps 7.9 → 5.9 vs the page-tree baseline; +10.8% computation for induction (its App. D). In ASI’s re-run: 5.9 steps vs 5.6 plain	Prompt grows
Beyond Browsing (Song et al., 2025)	API calls and API documentation in context, with browsing as fallback	WebArena, GPT-4o: success 14.8 → 38.9% (hybrid), steps 8.4 → 8.5, \$0.1 → \$1.4 per task	API documents lengthen the prompt
ComputerRL (Lai et al., 2026)	An API-GUI action space	OSWorld, prompted GPT-4o: success 11.2 → 26.2%	No step, time or \$ figures
CoAct-1 (Song et al., 2026)	An orchestrator hands sub-tasks to a coding agent or a GUI agent	OSWorld-Verified: success 53.07 → 59.93% vs GTA-1-7B with o3; 10.15 vs 15.22 steps per passed task	Step budget behind 10.15 ambiguous
OSWorld-Human (Abhyankar, Qi & Zhang, 2026)	Human reference trajectories, grouped by screenshot	Steps per trajectory, single → grouped: Chrome 5.8 → 4.3 (1.35×) to Calc 13.2 → 4.5 (2.93×) (calc.); 369 tasks	A human-reference headroom estimate, not a method or a strict bound

B2 · Capping the steps: stopping rules and budgets

behind page 4 · term N capped · a stop can end attempts that would have succeeded (R_m(p) ↓) · calc. = our arithmetic

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
EET (Guo et al., 2026)	Stops a run early when past experience predicts no gain	SWE-bench Verified, GPT-5-mini, Agentless: resolved 33.2 → 41.0%, benchmark cost \$13.77 → \$6.18, API calls −26.4%; six-configuration average cost −31.8%	Offline experience base; one configuration −0.2 points
Runaway is ashamed (Lu et al., 2025b)	A verifier decides when to exit	ALFWorld 134 tasks, Llama-3.1-70B: steps 19.0 → 13.4, success 76.1 → 70.2%	Success falls
Budget Tracker (Liu et al., 2026a)	Tells the agent its remaining tool budget	BrowseComp, Gemini-2.5-Pro: 12.8% at budget 10 vs 12.6% at 100; cost per question 9.9¢ → 6.8¢ (−31.3%)	Tool calls priced at a flat \$0.001
Efficient Agents (Wang et al., 2025a)	A cheaper agent configuration	GAIA, 165 tasks (calc.): \$0.398 → \$0.285 per task, 53.33 → 51.52%; one run	Baseline configuration not stated
TTI (counterpoint) (Shen et al., 2025)	Trains the agent to take more steps (interaction scaling)	WebVoyager 427 tasks: 55.8 → 64.8% (zero-shot → TTI); WebArena 812: 18.3 → 26.1%; Gemma 3 12B. Longer thinking at equal compute gave under 3%	N grows by design; no cost figures

B3 · Agent-aware serving: more systems

behind page 5 · terms t^{queue} and t^{prefill} · self-hosted engines; all figures are throughput or latency under load, none reports task quality unless stated

SYSTEM	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	BASELINE
SGLang (Zheng et al., 2024)	Prefix sharing in a radix tree of KV caches	Up to 6.4× throughput, 3.7× lower latency; Llama-7B on one A10G; 12 workloads incl. ReAct and generative agents (replayed)	vLLM v0.2.5, Guidance, LMQL (which gives the maximum is not stated)
Preble (Srivatsa et al., 2025)	Prompt-sharing-aware scheduling across GPUs	1.5–14.5× lower mean and 2–10× lower p99 latency; tool use, embodied agent, program generation, video and long-document QA; Mistral 7B, Llama-3 70B	Data-parallel SGLang
Parrot (Lin et al., 2024)	Semantic variables expose a program’s structure to the server	Up to 11.7× lower end-to-end latency; MetaGPT multi-agent programming, LLaMA 13B, one A100	Latency-centric vLLM
KVFlow (Pan et al., 2025)	Workflow-aware cache eviction and prefetching	1.24× vs SGLang with hierarchical cache (camera-ready, Qwen2.5-32B, one H100); the 1.83× headline is from the v1 A10G set-up only	SGLang HiCache
Helium (Wadlom et al., 2026)	Workflow-level optimisation of batched agent queries	Up to 1.56× vs KVFlow (end-to-end latency of a batch, optimisation included)	KVFlow
KVCOMM (Ye et al., 2025)	Reuses KV caches across agents with offset correction	First-token time 7.82× lower at the fifth agent (428.6 → 54.8 ms); Llama-3.1-8B, one H100, HuggingFace	Full prefill
DroidSpeak (Liu et al., 2026b)	Shares KV caches between fine-tuned variants of one model	Prefill 1.7–3.1× faster (average 2.1×); eight model pairs, three datasets each	Full prefill
AsymCache (Shi et al., 2026a)	Asymmetric cache eviction for multi-session serving	With Continuum, average job latency 4.4–18.1% below Continuum alone; alone 0.4–4.2% below vLLM-LRU (BFCL web search, no task count); its own headline ranges are inconsistent	Continuum; vLLM-LRU
CacheBlend (Yao et al., 2025)	Blends precomputed chunk caches, recomputing a few tokens	First-token time 2.2–3.3× lower, quality within 0.02 F1 — RAG, not agents	Full recompute

B4 · Prefill and context: more rows, with full conditions

behind pages 6 and 11 · terms |o_{a,i}|, g̃ and t^prefill · calc. = our arithmetic

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
FocusAgent, other settings (Kerboua et al., 2026)	Reader model GPT-5-mini instead of GPT-4.1-mini	WorkArena L1: input cost −31.5% (calc.) at 53.2% success; WebArena 381 tasks: 36.5 → 39.6%, −21.7%. Input tokens only are priced	No latency for this reader
LineRetriever (Kerboua et al., 2025)	A reader model keeps whole lines of the page tree	Observation −61% on WorkArena L1 (330 runs), success 52.7 → 44.8%	Success falls
Prune4Web (Zhang et al., 2026c)	A model writes a scoring program; the top 20 elements go to the grounder	25–50× fewer candidate elements; grounding accuracy 46.8 → 88.28% with ground-truth sub-tasks (1,101 steps)	Grounding, not task success; no latency measured
SimpAgent (Chen et al., 2025)	Masks irrelevant screenshot regions and compresses history	FLOPs per step 11.90 → 8.71 T; offline step success 69.0 → 71.3% (AITW), Qwen2-VL-2B	Offline steps
ScreenSeeker (counter) (Li et al., 2025)	A planner searches the screen recursively for the target	Grounding 18.9 → 48.1% on 1,581 ScreenSpot-Pro instructions, GPT-4o planner	More model calls per step
Observation masking, other models (Lindenbauer et al., 2025)	As on page 11	Gemini 2.5 Flash \$0.41 → \$0.18, 32.8 → 35.6%; with thinking 40.4 → 36.4% (significant); Qwen3-32B \$1.12 → \$0.55, 17.0 → 15.0%	Success can fall
AgentDiet, per task (Xiao et al., 2026)	As on page 11	\$0.535 → \$0.422 (Claude, SWE-bench Verified), \$0.701 → \$0.449 (Gemini, Multi-SWE-bench Flash); cached-input discount applied	Reducer 5.5–11.8% of the original cost
FireAct (Chen et al., 2023)	A fine-tuned GPT-3.5 drops the few-shot examples from every prompt	HotpotQA 500 questions: 9.0 → 2.7 s per trial, \$2.6e-3 → \$2.2e-3, EM 31.4 → 39.2	Price per token ×8 (c_κ(μ) ↑)
TokenPilot, mixed sessions (Xu et al., 2026a)	As on page 12	123 tasks shuffled into 10 sessions: \$21.16 → \$4.83 (−77%, calc.), score 68.77 → 69.10	Partial-credit scores

B4 · Decoding: more rows, with full conditions

behind page 7 · terms n^out and TPOT · vendor = the vendor’s own statement

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
Think Twice, Click Once (Tang et al., 2025)	Switches between a fast and a slow grounding mode	Time per sample 3.2 → 2.6 s, accuracy 74.8 → 77.4%; slow mode only 5.4 s — 1,272 samples, Qwen2-VL-2B, same model with and without its switch threshold	Grounding only
Does CoT help mobile agents? (Zhang et al., 2026d)	Caps the reasoning tokens	AndroidControl, GLM-4.6V: action accuracy 71.55% at 128 tokens vs 70.41% unlimited and 70.32% without reasoning	Sample size not stated
TSDS (Farzaneh & Simeone, 2026)	Short edge thinking, deferral to the cloud when unsure	MBPP (257 problems, single step): edge thinking tokens 1,183 → 422, reward 0.719 → 0.714	Cloud deferral rate 0.346; cloud tokens not counted
GUI-KV (Huang et al., 2026a)	Keeps 40% of the KV cache per layer (5 screenshots in context, UI-TARS-1.5-7B)	Decoding MFLOPs per token −38.9%, offline step accuracy 17.5 → 21.6% (AgentNetBench); OSWorld 26.0 → 25.1%	No wall-clock
Agent-X (Chung et al., 2026)	A precomputed prompt-prefix cache and speculative decoding on a Mac mini	Task time 1.61× faster (prefill 1.97×, decode 1.73×) on 1,022 TinyAgent examples, Mac mini M4 Pro	On-device only
Effort guidance (vendor) (Anthropic, 2026a)	Medium instead of high reasoning effort	“Roughly half the output tokens of high” with close to the best success, and the same success once retries are allowed — internal UI-automation suite, no counts	Vendor statement
Ultrafast (vendor) (OpenAI, 2026; Cerebras, 2026)	GPT-5.6 Sol on Cerebras hardware	Up to 14× faster than Standard, up to 750 output tokens per second; limited preview, no price	OpenAI vendor statements; a hardware partner’s 6-task chart is marketing, not cited

B5 · Environment and overlap: more rows, with full conditions

behind pages 8–10 · terms E_i and T_saving · calc. = our arithmetic

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
Skim, details (Wong et al., 2026)	As on page 8; a local Qwen2.5-14B runs the fast path and the verifier	Accuracy vs the default agent: 40.6 vs 37.6 (WebVoyager agent), 52.0 vs 49.6 (AgentOccam), 45.6 vs 45.0 (BrowserUse); verifier precision 82.0%, recall 86.2%	Site profiling 6–24 s per site
TPS-Bench (Xu et al., 2026b)	RL-trained scheduling of dependent tool calls	TPS-Bench-Hard, 100 tasks, Qwen3-1.7B: time 42.0 → 34.8 s per task (–17.1%, calc.), completion 26.75 → 35.17%	Its text gives 14% and 6% (from v1)
AsyncLM (Gim et al., 2024)	Asynchronous function calls with interrupts	Local fine-tuned Llama-3.2 on one RTX 4090: 1.6–2.4× lower latency than sequential calls, 1.23–1.5× than parallel calls (calc.; first to last generated token); cloud figures emulated	Emulated for GPT-4o
Speculative tool calls (Nichols et al., 2025)	Client- and engine-side speculation of tool calls	6–21% of time saved with synthetic tool latencies and cached tool outputs; BFCL, 32 tasks per agent	Synthetic latencies
DSP (Guan et al., 2026)	Learns how many steps ahead to speculate	OpenAGI 312 tasks, GPT-4.1-mini: –37.09% latency at +62.99% cost; the cost is against the sequential cost of both agents	Cost vs a target-only agent not reported
Speculate with memory (Li et al., 2026c)	Guesses from a memory of past runs	Analytical peak 1.128×; HotpotQA 1.05× — replay estimates	Not end-to-end
AgenticCache (Kim et al., 2026)	Cached plans for embodied agents	TDW-COOK, 18 episodes: 12.86 → 1.75 h, \$21.0 → \$4.4, success 94.44 → 100%; TDW-MAT 1.86× (calc.)	Embodied simulator
IdleSpec (counter) (Choi et al., 2026)	Drafts plans while tools run	GAIA 165 + FRAMES 50, Gemini-2.5-Flash: accuracy 50.5 → 55.6%, wall-clock about unchanged; output tokens +58% (calc., Qwen3.5-4B)	Tool-wait overlap spent on accuracy (R_m(p) ↑), not on time
ThunderAgent (Kang et al., 2026)	Prepares the sandbox while the program waits for GPU memory (overlap, T_saving)	Environment time 4.8 → 0.3 s — OpenHands rollouts, self-hosted (its Fig. 6a)	Throughput system; no task success
TClone · DeltaBox (Huang et al., 2026b; Dong et al., 2026)	Fast container clone; millisecond checkpoint and rollback	Clone for one OSWorld branch point 18.2 → 3.7 s vs KVM; rollback 1.86 ms on the fast path (9.29 ms slow path)	Cost of branching E_i

B6 · Prices by billing class and tier, per vendor

behind pages 12–13 · vendor price pages and documentation, read 28 September 2026 · ratios are our arithmetic (calc.)

VENDOR, LATEST MODEL	UNCACHED → CACHE HIT (PER 1M INPUT TOKENS)	CACHE WRITE, STORAGE, LIFETIME	TIERS
DeepSeek-V4.1-Flash (DeepSeek, 2026)	\$0.30 → \$0.006 (0.02×), peak; off-peak half	none listed	—
Anthropic, Claude Opus 5.5 (Anthropic, 2026b)	\$4 → \$0.20 (0.05×); Fable 5.1 and Mythos 5.1 0.025×; other Claude models 0.1×	1.25× for 5 min, 2× for 1 h; refreshed free on each use; a change of speed, tools, thinking or effort invalidates parts of the cache	Fast mode 2× (\$8 / \$40), up to 2.5× output speed
OpenAI, GPT-6 Sol / Luna (OpenAI, 2026)	\$2 → \$0.20; \$0.10 → \$0.01 (0.1×)	1.25× from GPT-5.6 on; at least 30 min	Fast 2×; Flex and Batch 0.5×
Google, Gemini 3.8 Flash (Google, 2026)	\$0.75 → \$0.075 (0.1×), to 31 Dec 2026	storage \$0.50 per 1M tokens per hour	Priority 1.8× on every listed price (text says 75–100%), Preview
xAI, Grok 4.7 (xAI, 2026)	\$2 → \$0.50 (0.25×)	none listed	—
Microsoft Foundry (Microsoft, 2026a)	—	—	Priority processing 1.75–2.5× (calc., from the pricing page's HTML)

Worked example (OpenAI, GPT-5.6 and later): one write and one read cost $1.25 + 0.1 = 1.35\times$ against $2\times$ uncached; one write and nine reads cost $2.15\times$ against $10\times$ (calc.).

B7 · A cheaper model per call: more rows, with full conditions

behind page 13 · term $c_k(\mu_{ij})$ · calc. = our arithmetic

WORK	MECHANISM	MEASURED EFFECT, WITH ITS CONDITIONS	LEFT OUT, OR GROWS
StepWise, web (Wei et al., 2026)	gpt-oss-20b by default, GPT-5.2 when stuck	WebArena-Verified (~812 tasks, calc.): \$0.335 → \$0.211 per task, 60.1 → 57.8%, time per request 19.6 → 12.2 s; local 2×H100 timing	Large model on 66.9% of steps
BoPO (Zhang et al., 2026b)	A router under a cap on large-model calls	AppWorld 168 tasks, GPT-4.1 / GPT-4.1 mini, 3 seeds: large calls per task 18.75 → 13.2 (−29.6%, calc.), success 66.7 → 66.5%	Router cost excluded
Agentic plan caching, per workload (Zhang et al., 2025)	As on page 13	FinanceBench \$4.03 → \$1.86, 91.0 → 85.5%; QASPER \$2.14 → \$0.78; GAIA \$69.02 → \$16.27, 37.58 → 36.97%	GAIA task count not stated
RouteLLM (Ong et al., 2025)	A learned router between a strong and a weak model	3.66× fewer GPT-4 calls than a random router at equal quality (MT-Bench); 1.41× (MMLU), 1.49× (GSM8K)	Single-turn; not against always-GPT-4
FrugalGPT (Chen, Zaharia & Zou, 2024)	A cascade of models with a stopping scorer	Up to 98% lower cost at equal accuracy on one single-query dataset (HEADLINES)	Single-turn

B8 · Cost per success: the inputs, the formula and the rounding

behind page 14 · v = cost per attempt ÷ success rate (Cost-of-Pass, Eq. 2) · all results are calc. unless stated

WORK	COST PER ATTEMPT (AS PRINTED)	SUCCESS RATE	COST PER SUCCESS (CALC.)	NOTE
AXIS vs UFO (Lu et al., 2025a)	\$0.4 → \$0.2	52.0 → 84.0%	\$0.77 → \$0.24	costs printed to one decimal
W&D (Lin et al., 2026)	\$1.025 → \$0.657 (\$102.5, \$65.7 per 100)	66 → 68%	\$1.55 → \$0.97	includes search and scraping fees
OSWorld 2.0, Opus 4.8 (XLANG Lab, 2026)	~\$76.1 → ~\$72.4	18.5 → 20.6%	\$411 → \$351	Opus 4.7: \$258 → \$185
Beyond Browsing (Song et al., 2025)	\$0.1 · \$1.2 · \$1.4	14.8 · 29.2 · 38.9%	\$0.68 · \$4.11 · \$3.60	browsing · API · hybrid; one-decimal costs, so the ratio is 3–11×
BATS vs ReAct@100 (Liu et al., 2026a)	9.9¢ → \$1.1	12.6 → 24.6%	\$0.79 → \$4.47	BrowseComp, Gemini-2.5-Pro
ActionEngine (Zhong et al., 2026)	\$0.71 → \$0.06	66 → 95%	\$1.08 → \$0.06	different models on the two sides
StepWise, OSWorld (Wei et al., 2026)	\$0.881 → \$0.224	58.1 → 55.4%	\$1.52 → \$0.40	
WebRouter (Li et al., 2026b)	\$0.98 → \$0.12	86.1 → 82.3%	\$1.14 → \$0.15	
Fara-7B vs computer-use-preview (Awadallah et al., 2025)	\$0.913 → \$0.025	70.9 → 73.5%	\$1.29 → \$0.03	WebVoyager
AI Agents That Matter (Kapoor et al., 2025)	LATS \$134.50 vs warming \$2.45 (164-problem run totals)	88.0 vs 93.2%	stated totals, not per success	HumanEval, GPT-4

With a set-up cost (page 14): $v_n = C_{\text{setup}}/n + v$. Two retained works print both parts. EET’s experience base cost \$4.3 once, against a benchmark run of \$13.77 → \$6.18 (500 tasks), so it pays off within the first run: \$10.48 with the set-up charged to it (calc.). SpeedRunner already spreads its skill-inducer calls over the runs they serve. The page-3 systems print their set-up in minutes, tokens or GPT-4o dollars, never next to a per-task dollar figure.

B9 · Works held back or dropped, and why

the source-credibility rule of slides/references.md · (iii) = numbers from a public benchmark with the task count and model stated

STATUS	WORKS	REASON
Held back: condition (iii) not met as written, pending Edwin’s decision	SpecHop (arXiv:2605.21965), PASTE (2603.18897), AOSpec (2608.00881), TAB, CacheScout (2608.14624), DualPath, PBKV, AgentKVShift, SpecBox (2607.23933), LLM-Tool Compiler, Cordon (2606.17573), GoClick, BAVT; DualSpec’s XBench row; Continuum’s real-run 8.18×	No task count, internal traces, or an unnamed model; their numbers stay off the pages
Shown with a flag, pending the same decision	Skim (“300+” tasks, page 8); ARES (~129 WebArena tasks, calc.), Overthinking (SWE-bench Verified), StepWise (OSWorld), Fara-7B (WebVoyager): task counts not printed; the author-built test suites of Atomix (leak test), Safe to Resume? (96 workflows) and Ghost Tool Calls (30 tasks)	Counts derived by us, implied by the benchmark, or from suites the authors built; labelled on the pages
Dropped: fail the rule	Octopus v2, GPA, SkillDroid (qualitative only), AgentServe, Stateful Inference (Norgren), Cost-Aware Speculative Execution (Fareed), Dynamic ReAct, AdaGUI-R1, a speculative-tools GitHub repository	In-house sets without counts, single authors without measurements, no institution, or unreadable
Could not be read or found	OS-Catalyst; ToolSEE; the Self-Guide “+22%”, VITA-VLA “–76%” and “2,000–8,000 schema tokens” figures	The figures are not in the sources
Off-topic	SEAR, DREAM-Chunk (robotics); VITA-VLA, m2mKD (not LLM agents); SRMT (not an LLM); MemRefine (offline storage); DARE (math reasoning); Self-Guide (no efficiency measure)	No Part 1 term
Qualitative only	ECLAIR, AgentRR, Signal-Driven Observation, the small-models position paper, the LATM / DiLogics / ALLOY / Voyager / WebAgent lineage, RAC, Revisable by Design, AAPT, Qwen-UI-Agent, Log2Plan, ConServe, TOPAS, SmoothAgent, AAFLOW+ (modelled), PEEK, PANDO (flagged), vendor cache marketing	No usable measured number, or internal inconsistencies
Numbers replaced by corrections	SkillDroid 2.4×; Hajimiri v1; KVFlow 1.83× (absent from the camera-ready body); AgentServe 2.8× (against llama.cpp); RouteLLM 3.66× “vs GPT-4”; UFO2 “51.5% lower inference cost”; WALT “1.3–1.4× on average”	Superseded by the source checks

B10 · What no retained work measures yet, by term

gaps in the evidence, not claims of a research gap · the one-line versions are in each page’s foot

TERM	NOT YET MEASURED
J_i (compile, replay)	Replay under live application change; a vendor replay or repair rate with a sample; cost per success with set-up included; a multi-page enterprise form with a final submit
t^{queue}	Its share of task time for API agents; measured latency of API priority tiers (only classes or targets are published)
$t^{\text{prefill}}, o_{a,i} $	Task time with caching on; how pruning interacts with prefix caching; the load time of cache-hit tokens, which Part 1 has no term for
n^{out} , TPOT	The net effect of per-step effort changes that invalidate the message cache; independent task-level timing of fast tiers
E_i	Page-load and wait time in write-heavy web workflows (Skim is read-only); wall-clock of batched actions
T_{saving}	Speculation on a live web GUI with side effects held until commit; cost per success of any speculation method; several speed figures are analytical or replayed
$\bar{g}, N^2$	The N² token volume together with the cache hit rate in a GUI agent; structural removal is shown only by compiled plans; eviction (TokenPilot) may change the shape but is not measured as such
$n^{\kappa}, x_{\text{env}}$	A GUI or browser agent’s tokens split by billing class, cache writes included (Don’t Break the Cache covers a web-search agent); environment hours priced beyond a vendor list price (coding agents only in TraceLab)
$c_{\kappa}(\mu_{ij})$	The router’s own cost; small-model losses on enterprise forms; routing with cached prices
$R_m(p)$, set-up	Cost per success reported by the papers themselves; time, money and success, set-up included, on one web or GUI benchmark